Naija Prime School

Sprint 7 — Store & Inventory Management

Suppliers · Catalog · Stock movements · Storekeeper dashboard

Long-form implementation walk-through

Author: Benjamin Fadina

Branch: sprint/7-store-inventory

Built on: Sprints 1–6 (identity, academic domain, students & parents, attendance, results & report cards, pupil photos, fees & bursar workflows)

Stack: .NET 10, Blazor Web App (Auto), EF Core 10, SQL Server, Radzen Blazor

Editor: Visual Studio Code with the C# Dev Kit

Repository: https://github.com/benjaminsqlserver/NaijaPrimeSchool

Contents

1. Sprint 7 in context

Sprint 7 builds the storekeeper's workspace. The bursar's side of the financial coin closed in sprint 6 — fee schedules, invoices, receipts, and the dashboard that tells the bursar how much money is coming in and how much is still owed. Sprint 7 turns its attention to the other side of the same building: the store room. Every textbook the school issues to a pupil, every uniform handed across the counter, every bag of rice the kitchen draws down, every reams of A4 the office consumes, every football the games master collects — those movements all flow through a single audit table that this sprint introduces.

Functionally the sprint introduces three layered concepts. The Supplier directory captures the vendors the school buys from. The StoreItem catalog lists every distinct article the storekeeper tracks, with category, unit of measure, reorder level, and a running on-hand quantity. The StockMovement table is the audit trail: every receipt, issuance, return, write-off, or adjustment is one row, and the on-hand quantity on the parent StoreItem is brought into line with the stream of movements via a service-layer helper.

Once this sprint ships, the SchoolStoreKeeper role — seeded back in sprint 1 but unused since — finally has a workspace. The previously-disabled 'Store & Inventory' navigation placeholder lights up with six items: a dashboard summarising stock value and low-stock alerts, the catalog, item detail, movement log, record-new-movement form, and supplier directory.

This document is a long-form implementation guide written in the tone of the sprint 6 guide. An engineer who has read the earlier guides and has the codebase checked out can recreate every change here without referring to the diff.

1.1 Where this sits relative to sprint 6

Every load-bearing piece of the earlier sprints is reused:

- BaseEntity — every new entity inherits Guid Id, IAuditable, and ISoftDelete from it.
- ApplicationDbContext.SaveChanges — the override stamps audit columns and rewrites Delete to IsDeleted = true. Every Supplier, StoreItem and StockMovement therefore inherits the same audit and soft-delete machinery as everything else.
- Global query filters — every new entity declares HasQueryFilter(x => !x.IsDeleted), so deleted rows vanish from ordinary queries automatically.
- OperationResult / OperationResult<T> — every new service uses this for predictable success/failure responses.
- ILookupService — already had twenty-four methods. Sprint 7 adds five more (item categories, units of measure, stock movement types, active suppliers, and a typeahead for store items) without rewriting the existing ones.
- Student, SchoolClass — pick up StockIssuances back-references so the storekeeper can see, on a pupil's profile (in a future sprint), every item ever issued to them. No scalar columns change on either table.
- StudentAvatar (sprint 5b) — already lives in Components.Shared and is reused on the movement log when an item is issued to a pupil.

1.2 Functional scope delivered

1. Maintain a Supplier directory — name, contact, phone, email, address, notes. Soft delete refused once a purchase has been recorded against the supplier; deactivate instead.
2. Maintain the StoreItem catalog. Categories and units of measure are seeded lookup tables. Each item carries a reorder level and a running QuantityOnHand. Optional SKU is a unique filtered index.
3. Open a new item with an opening balance — the create form lets the storekeeper enter an opening quantity and unit cost, which the service persists as a single 'Opening Balance' stock movement and the item's on-hand value.
4. Record every stock movement: Purchase, Return, Adjustment In, Opening Balance (inbound); Issue, Write-off, Adjustment Out (outbound). Each movement carries quantity, unit cost, total cost, optional reference (PO number, requisition number) and free-text notes.
5. Pair an issuance with its recipient: a pupil, a class, or a staff member. A purchase pairs with a supplier. The service enforces at most one recipient per movement.
6. Watch the dashboard: items in catalog, items below reorder, stock value, this-month inbound / outbound counts, low-stock list, recent movements, value by category.
7. Soft-delete items, suppliers, and movements with guards: cannot delete an item with movement history, cannot delete a supplier with purchase history, cannot record an outbound movement larger than the on-hand quantity. Deleting a movement reverses its effect on the running balance.

1.3 Non-goals deliberately deferred

- Multi-location inventory. The school is single-site; a future Store table per physical location would slot in without redesigning anything else.
- Batch / lot tracking. Some items (food, medicines) would benefit from per-batch expiry dates. The current model holds one running balance per item; a future StoreItemBatch table would be additive.
- Purchase orders. The Reference column on a StockMovement carries a PO number today; a dedicated PurchaseOrder table with approvals and partial receipts is its own sprint.
- Pupil-facing returns (the family bringing the textbook back at year end). The Return movement type captures it on the storekeeper side; the parent portal sprint will add the pupil-facing surface.
- Tying issuances to the bursar's invoices. A school that charges separately for uniforms could link an Issue movement to a Sprint-6 InvoiceLine. The FK is not in the schema today because we have not yet seen a school that wants that link automated.
- Reorder workflow. Today the dashboard surfaces items below reorder; clicking through to 'create a purchase from this list' is a future workflow.

1.4 Scale of the sprint

By the numbers:

- 6 new domain entities under src/NaijaPrimeSchool.Domain/Inventory/.
- 2 collection navigations on existing entities (Student, SchoolClass).
- 4 DTO files under src/NaijaPrimeSchool.Application/Inventory/Dtos/.
- 3 new service contracts under src/NaijaPrimeSchool.Application/Inventory/.
- 3 service implementations under src/NaijaPrimeSchool.Infrastructure/Services/.
- 5 new methods on ILookupService (and the matching LookupService).

- 1 EF Core migration introducing 6 new tables and the indexes that go with them.
- 1 DatabaseInitializer extension seeding ItemCategories, UnitsOfMeasure, and StockMovementTypes.
- 6 Razor pages under src/NaijaPrimeSchool.Web/Components/Pages/Inventory/.
- 1 navigation menu rewrite: the previously-disabled 'Store & Inventory' placeholder is replaced with a six-item panel gated to SuperAdmin + HeadTeacher + SchoolStoreKeeper.

The code follows the patterns already accepted in sprints 1–6.

2. Design decisions and trade-offs

2.1 Movement-log as the source of truth

Every change to stock — in or out — is one row in StockMovement. StoreItem.QuantityOnHand is a cached running total that the service keeps in agreement with the movement stream. The alternative — calculate on-hand from a SUM(Direction * Quantity) every time the catalog renders — is correct but slow. Persisting the running balance and updating it inside the same SaveChanges that creates the movement gives us:

- Constant-time list queries: the catalog reads QuantityOnHand directly without aggregating thousands of movements.
- Atomic correctness: a movement insert and the balance update are in the same transaction, so the two cannot diverge.
- A reversible audit log: soft-deleting a movement reverses its contribution to the running balance, which makes 'undo a wrongly-keyed receipt' a one-click operation.

2.2 No enums — three lookup tables

The rule from earlier sprints holds. ItemCategory, UnitOfMeasure, and StockMovementType are all proper entities derived from BaseEntity, seeded on first run, and editable from the database without a redeploy. StockMovementType carries a Direction column (+1 = inbound, -1 = outbound) that the service multiplies Quantity by to roll up the on-hand figure — so a future 'Loan Out' or 'Loan Back' row could be added without changing any service code.

2.3 Decimal precision

Quantities are decimal(14,3) — three decimal places of resolution lets the school track partial kilograms of rice or litres of cleaning fluid without rounding. Unit cost is decimal(12,2); total cost is decimal(14,2). These match the decimal precisions used in the finance domain in sprint 6.

2.4 Movement numbering

MovementNumber follows the same year-prefixed pattern as Sprint-6 invoices and receipts: NPS/STK/<year>/<4-digit-sequence>. The service computes the next sequence by reading the largest existing number for the year and adding one. Numbers are guaranteed unique by a database index, and auditable across years.

2.5 At most one recipient per movement

An Issue movement can be paired with a pupil, a class, or a staff member — but not two of them. Three nullable FKs (IssuedToStudentId, IssuedToSchoolClassId, IssuedToUserId) give us flexibility without the alternative shapes (polymorphic foreign keys, JSON columns) that would make the audit trail brittle. The service rejects a request with more than one set. Leaving all three null is allowed and represents 'general consumption' — e.g. the cleaning team drew a bag of soap powder for the school.

2.6 Soft delete plus operation guards

- Supplier cannot be deleted if it has purchase history. Deactivate instead.
- StoreItem cannot be deleted if it has movement history. Deactivate instead.

- Outbound StockMovement is refused if Quantity exceeds the item's on-hand balance.
- Soft-deleting a StockMovement reverses the running balance on its item, so the audit history can be hidden without leaving a phantom quantity behind.

2.7 Unique indexes

- Unique on each lookup's Name and Code (4 unique indexes across ItemCategory, UnitOfMeasure, StockMovementType).
- Filtered unique on StoreItem.Sku WHERE [Sku] IS NOT NULL — items don't have to carry a SKU but if they do, it must be unique.
- Unique on StockMovement.MovementNumber — auditable.

2.8 Inline forms, the same UI rhythm

Every Razor page in this sprint follows the same shape as the earlier sprints: a filter card on top, a paged grid in the middle, an inline RadzenCard form below the grid for new-or-edit, and a confirm-then-act dialog for destructive operations. Two pages — CreateStoreItem and RecordStockMovement — are dedicated full-page forms because their workflows have enough fields and side-effects that an inline form would have been cramped. The record-movement page also flips its lower half between an inbound 'received from supplier' panel and an outbound 'issued to recipient' panel based on the chosen movement type's Direction.

3. The Domain layer in full

3.1 Folder layout

```
src/NaijaPrimeSchool.Domain/
└── Inventory/                      <- (new in sprint 7)
    ├── ItemCategory.cs             <- lookup
    ├── UnitOfMeasure.cs            <- lookup
    ├── StockMovementType.cs        <- lookup with Direction
    ├── Supplier.cs                 <- vendor entity
    ├── StoreItem.cs                <- catalog entry
    └── StockMovement.cs            <- audit row
```

3.2 The three lookup entities

3.2.1 ItemCategory.cs

Listing — src/NaijaPrimeSchool.Domain/Inventory/ItemCategory.cs

```
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Inventory;

public class ItemCategory : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public string Code { get; set; } = string.Empty;
    public int DisplayOrder { get; set; }

    public ICollection<StoreItem> Items { get; set; } = [];
}
```

3.2.2 UnitOfMeasure.cs

Listing — src/NaijaPrimeSchool.Domain/Inventory/UnitOfMeasure.cs

```
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Inventory;

public class UnitOfMeasure : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public string Code { get; set; } = string.Empty;
    public int DisplayOrder { get; set; }

    public ICollection<StoreItem> Items { get; set; } = [];
}
```

3.2.3 StockMovementType.cs

Listing — src/NaijaPrimeSchool.Domain/Inventory/StockMovementType.cs

```
using NaijaPrimeSchool.Domain.Common;
```

```
namespace NaijaPrimeSchool.Domain.Inventory;

public class StockMovementType : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public string Code { get; set; } = string.Empty;

    // +1 = inbound (stock goes up), -1 = outbound (stock goes down).
    // The Direction column lets the service layer compute net stock from
    // a stream of movements without keying off the row name.
    public int Direction { get; set; }

    public int DisplayOrder { get; set; }

    public ICollection<StockMovement> Movements { get; set; } = [];
}
```

3.3 The three core entities

3.3.1 Supplier.cs

Listing — src/NaijaPrimeSchool.Domain/Inventory/Supplier.cs

```
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Inventory;

public class Supplier : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public string? ContactName { get; set; }
    public string? Phone { get; set; }
    public string? Email { get; set; }
    public string? Address { get; set; }
    public string? Notes { get; set; }

    public bool IsActive { get; set; } = true;

    public ICollection<StockMovement> Purchases { get; set; } = [];
}
```

3.3.2 StoreItem.cs

Listing — src/NaijaPrimeSchool.Domain/Inventory/StoreItem.cs

```
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Inventory;

public class StoreItem : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public string? Sku { get; set; }
```

```
    public string? Description { get; set; }

    public Guid ItemCategoryId { get; set; }
    public ItemCategory? ItemCategory { get; set; }

    public Guid UnitOfMeasureId { get; set; }
    public UnitOfMeasure? UnitOfMeasure { get; set; }

    public decimal QuantityOnHand { get; set; }
    public decimal ReorderLevel { get; set; }

    public decimal LastUnitCost { get; set; }

    public bool IsActive { get; set; } = true;

    public ICollection<StockMovement> Movements { get; set; } = [];
}
```

QuantityOnHand and LastUnitCost are persisted convenience columns kept in agreement with the movement stream by the service. The catalog list page reads these directly so it can render thousands of items without aggregating millions of movements.

3.3.3 StockMovement.cs

Listing — src/NaijaPrimeSchool.Domain/Inventory/StockMovement.cs

```
using NaijaPrimeSchool.Domain.Academics;
using NaijaPrimeSchool.Domain.Common;
using NaijaPrimeSchool.Domain.Family;
using NaijaPrimeSchool.Domain.Identity;

namespace NaijaPrimeSchool.Domain.Inventory;

public class StockMovement : BaseEntity
{
    public Guid StoreItemId { get; set; }
    public StoreItem? StoreItem { get; set; }

    public Guid StockMovementTypeId { get; set; }
    public StockMovementType? StockMovementType { get; set; }

    public string MovementNumber { get; set; } = string.Empty;
    public DateOnly MovedOn { get; set; }

    public decimal Quantity { get; set; }
    public decimal UnitCost { get; set; }
    public decimal TotalCost { get; set; }

    public string? Reference { get; set; }
    public string? Notes { get; set; }

    // Optional counter-parties. At most one of the IssuedTo* should be set on
    // any one row; service-layer logic enforces the rule. ReceivedFromSupplier
    // is independent and used by purchase / return-to-vendor movements.
```

```
    public Guid? ReceivedFromSupplierId { get; set; }
    public Supplier? ReceivedFromSupplier { get; set; }

    public Guid? IssuedToStudentId { get; set; }
    public Student? IssuedToStudent { get; set; }

    public Guid? IssuedToSchoolClassId { get; set; }
    public SchoolClass? IssuedToSchoolClass { get; set; }

    public Guid? IssuedToUserId { get; set; }
    public ApplicationUser? IssuedToUser { get; set; }

    public Guid? PerformedById { get; set; }
    public ApplicationUser? PerformedBy { get; set; }
}
```

Five optional FKs: ReceivedFromSupplierId for purchases, plus the three IssuedTo* counter-parties for issuances, plus PerformedBy linking back to the storekeeper who keyed the row. All five OnDelete to SetNull so the soft-delete of any of those entities does not vaporise the audit history.

3.4 Relationships at a glance

```
       ItemCategory      UnitOfMeasure       StockMovementType
           \                 |                      |
            \                |                      |
             v 1..N          v 1..N                 v 1..N
                 StoreItem  --------+                |
                                    |                |
                                    v N              v
                               StockMovement --------+
                                    |
                                    +--- ReceivedFromSupplier (Supplier)
                                    +--- IssuedToStudent      (Student)
                                    +--- IssuedToSchoolClass  (SchoolClass)
                                    +--- IssuedToUser         (ApplicationUser)
                                    +--- PerformedBy          (ApplicationUser)
```

4. Application layer — DTOs and contracts

4.1 SupplierDtos.cs

Listing — src/NaijaPrimeSchool.Application/Inventory/Dtos/SupplierDtos.cs

```
using System.ComponentModel.DataAnnotations;

namespace NaijaPrimeSchool.Application.Inventory.Dtos;

public class SupplierDto
{
    public Guid Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string? ContactName { get; set; }
    public string? Phone { get; set; }
    public string? Email { get; set; }
    public string? Address { get; set; }
    public string? Notes { get; set; }
    public bool IsActive { get; set; }

    public int PurchaseCount { get; set; }
    public decimal TotalPurchased { get; set; }
}

public class CreateSupplierRequest
{
    [Required, StringLength(120)]
    public string Name { get; set; } = string.Empty;

    [StringLength(120)]
    public string? ContactName { get; set; }

    [StringLength(30)]
    public string? Phone { get; set; }

    [EmailAddress, StringLength(256)]
    public string? Email { get; set; }

    [StringLength(300)]
    public string? Address { get; set; }

    [StringLength(500)]
    public string? Notes { get; set; }

    public bool IsActive { get; set; } = true;
}

public class UpdateSupplierRequest
{
    public Guid Id { get; set; }

    [Required, StringLength(120)]
    public string Name { get; set; } = string.Empty;
```

```
    [StringLength(120)]
    public string? ContactName { get; set; }

    [StringLength(30)]
    public string? Phone { get; set; }

    [EmailAddress, StringLength(256)]
    public string? Email { get; set; }

    [StringLength(300)]
    public string? Address { get; set; }

    [StringLength(500)]
    public string? Notes { get; set; }
}

public class SupplierFilter
{
    public string? Search { get; set; }
    public bool? IsActive { get; set; }
}
```

4.2 StoreItemDtos.cs

Listing — src/NaijaPrimeSchool.Application/Inventory/Dtos/StoreItemDtos.cs

```
using System.ComponentModel.DataAnnotations;

namespace NaijaPrimeSchool.Application.Inventory.Dtos;

public class StoreItemDto
{
    public Guid Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string? Sku { get; set; }
    public string? Description { get; set; }

    public Guid ItemCategoryId { get; set; }
    public string ItemCategoryName { get; set; } = string.Empty;

    public Guid UnitOfMeasureId { get; set; }
    public string UnitOfMeasureName { get; set; } = string.Empty;
    public string UnitOfMeasureCode { get; set; } = string.Empty;

    public decimal QuantityOnHand { get; set; }
    public decimal ReorderLevel { get; set; }
    public decimal LastUnitCost { get; set; }
    public decimal StockValue => QuantityOnHand * LastUnitCost;

    public bool IsActive { get; set; }
    public bool IsBelowReorder => QuantityOnHand <= ReorderLevel;
}

public class CreateStoreItemRequest
```

```
{
    [Required, StringLength(120)]
    public string Name { get; set; } = string.Empty;

    [StringLength(60)]
    public string? Sku { get; set; }

    [StringLength(500)]
    public string? Description { get; set; }

    [Required] public Guid ItemCategoryId { get; set; }
    [Required] public Guid UnitOfMeasureId { get; set; }

    [Range(0.0, 999999999.0)]
    public decimal ReorderLevel { get; set; }

    [Range(0.0, 999999999.0)]
    public decimal OpeningQuantity { get; set; }

    [Range(0.0, 999999999.0)]
    public decimal OpeningUnitCost { get; set; }

    public bool IsActive { get; set; } = true;
}

public class UpdateStoreItemRequest
{
    public Guid Id { get; set; }

    [Required, StringLength(120)]
    public string Name { get; set; } = string.Empty;

    [StringLength(60)]
    public string? Sku { get; set; }

    [StringLength(500)]
    public string? Description { get; set; }

    [Required] public Guid ItemCategoryId { get; set; }
    [Required] public Guid UnitOfMeasureId { get; set; }

    [Range(0.0, 999999999.0)]
    public decimal ReorderLevel { get; set; }
}

public class StoreItemFilter
{
    public string? Search { get; set; }
    public Guid? ItemCategoryId { get; set; }
    public bool? IsActive { get; set; }
    public bool? OnlyBelowReorder { get; set; }
}
```

StoreItemDto carries the IsBelowReorder computed property so the catalog list can colour its badge without the consumer redoing the comparison. CreateStoreItemRequest carries an OpeningQuantity and OpeningUnitCost so a brand-new item can be seeded with stock-on-hand in one call.

4.3 StockMovementDtos.cs

Listing — src/NaijaPrimeSchool.Application/Inventory/Dtos/StockMovementDtos.cs

```
using System.ComponentModel.DataAnnotations;

namespace NaijaPrimeSchool.Application.Inventory.Dtos;

public class StockMovementDto
{
    public Guid Id { get; set; }

    public Guid StoreItemId { get; set; }
    public string StoreItemName { get; set; } = string.Empty;
    public string? StoreItemSku { get; set; }
    public string UnitOfMeasureCode { get; set; } = string.Empty;

    public Guid StockMovementTypeId { get; set; }
    public string StockMovementTypeName { get; set; } = string.Empty;
    public string StockMovementTypeCode { get; set; } = string.Empty;
    public int Direction { get; set; }

    public string MovementNumber { get; set; } = string.Empty;
    public DateOnly MovedOn { get; set; }

    public decimal Quantity { get; set; }
    public decimal UnitCost { get; set; }
    public decimal TotalCost { get; set; }

    public string? Reference { get; set; }
    public string? Notes { get; set; }

    public Guid? ReceivedFromSupplierId { get; set; }
    public string? ReceivedFromSupplierName { get; set; }

    public Guid? IssuedToStudentId { get; set; }
    public string? IssuedToStudentName { get; set; }
    public string? IssuedToStudentAdmissionNumber { get; set; }
    public string? IssuedToStudentPhotoUrl { get; set; }
    public string? IssuedToStudentFirstName { get; set; }
    public string? IssuedToStudentLastName { get; set; }

    public Guid? IssuedToSchoolClassId { get; set; }
    public string? IssuedToSchoolClassName { get; set; }

    public Guid? IssuedToUserId { get; set; }
    public string? IssuedToUserName { get; set; }

    public Guid? PerformedById { get; set; }
    public string? PerformedByName { get; set; }
}
```

```
public class RecordStockMovementRequest
{
    [Required] public Guid StoreItemId { get; set; }
    [Required] public Guid StockMovementTypeId { get; set; }
    [Required] public DateOnly MovedOn { get; set; }

    [Range(0.01, 999999999.0)]
    public decimal Quantity { get; set; }

    [Range(0.0, 999999999.0)]
    public decimal UnitCost { get; set; }

    [StringLength(120)]
    public string? Reference { get; set; }

    [StringLength(300)]
    public string? Notes { get; set; }

    public Guid? ReceivedFromSupplierId { get; set; }
    public Guid? IssuedToStudentId { get; set; }
    public Guid? IssuedToSchoolClassId { get; set; }
    public Guid? IssuedToUserId { get; set; }
    public Guid? PerformedById { get; set; }
}

public class StockMovementFilter
{
    public Guid? StoreItemId { get; set; }
    public Guid? StockMovementTypeId { get; set; }
    public Guid? SupplierId { get; set; }
    public int? Direction { get; set; }
    public DateOnly? FromDate { get; set; }
    public DateOnly? ToDate { get; set; }
    public string? Search { get; set; }
}
```

StockMovementDto carries the sprint-5b photo plumbing for issuances to pupils so the <StudentAvatar /> component renders a pupil's face on the movement log without a second round-trip.

4.4 StoreSummaryDtos.cs

Listing — src/NaijaPrimeSchool.Application/Inventory/Dtos/StoreSummaryDtos.cs

```
namespace NaijaPrimeSchool.Application.Inventory.Dtos;

public class StoreSummaryDto
{
    public int TotalItems { get; set; }
    public int ActiveItems { get; set; }
    public int ItemsBelowReorder { get; set; }
    public decimal StockValue { get; set; }
    public int MovementsThisMonth { get; set; }
    public int InboundThisMonth { get; set; }
```

```
    public int OutboundThisMonth { get; set; }

    public List<CategoryStockDto> ByCategory { get; set; } = [];
    public List<StoreItemDto> LowStockItems { get; set; } = [];
    public List<StockMovementDto> RecentMovements { get; set; } = [];
}

public class CategoryStockDto
{
    public Guid ItemCategoryId { get; set; }
    public string ItemCategoryName { get; set; } = string.Empty;
    public int ItemCount { get; set; }
    public decimal StockValue { get; set; }
}

public class StoreItemDetailDto
{
    public StoreItemDto Item { get; set; } = new();
    public List<StockMovementDto> Movements { get; set; } = [];
}
```

4.5 Service contracts

Listing — src/NaijaPrimeSchool.Application/Inventory/ISupplierService.cs

```
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Inventory.Dtos;

namespace NaijaPrimeSchool.Application.Inventory;

public interface ISupplierService
{
    Task<IReadOnlyList<SupplierDto>> ListAsync(SupplierFilter filter, CancellationToken ct
= default);
    Task<SupplierDto?> GetByIdAsync(Guid id, CancellationToken ct = default);

    Task<OperationResult<Guid>> CreateAsync(CreateSupplierRequest request,
CancellationToken ct = default);
    Task<OperationResult> UpdateAsync(UpdateSupplierRequest request, CancellationToken ct =
default);
    Task<OperationResult> SetActiveAsync(Guid id, bool isActive, CancellationToken ct =
default);
    Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct = default);
}
```

Listing — src/NaijaPrimeSchool.Application/Inventory/IStoreItemService.cs

```
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Inventory.Dtos;

namespace NaijaPrimeSchool.Application.Inventory;

public interface IStoreItemService
{
```

```
    Task<IReadOnlyList<StoreItemDto>> ListAsync(StoreItemFilter filter, CancellationToken
ct = default);
    Task<StoreItemDetailDto?> GetByIdAsync(Guid id, CancellationToken ct = default);

    Task<OperationResult<Guid>> CreateAsync(CreateStoreItemRequest request,
CancellationToken ct = default);
    Task<OperationResult> UpdateAsync(UpdateStoreItemRequest request, CancellationToken ct
= default);
    Task<OperationResult> SetActiveAsync(Guid id, bool isActive, CancellationToken ct =
default);
    Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct = default);
}
```

Listing — src/NaijaPrimeSchool.Application/Inventory/IStockMovementService.cs

```
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Inventory.Dtos;

namespace NaijaPrimeSchool.Application.Inventory;

public interface IStockMovementService
{
    Task<IReadOnlyList<StockMovementDto>> ListAsync(StockMovementFilter filter,
CancellationToken ct = default);
    Task<StockMovementDto?> GetByIdAsync(Guid id, CancellationToken ct = default);

    Task<OperationResult<Guid>> RecordAsync(RecordStockMovementRequest request,
CancellationToken ct = default);
    Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct = default);

    Task<StoreSummaryDto> GetStoreSummaryAsync(CancellationToken ct = default);
}
```

4.6 ILookupService extension

Excerpt — ILookupService.cs (sprint 7 additions)

```
    Task<IReadOnlyList<LookupDto>> GetItemCategoriesAsync(CancellationToken ct = default);
    Task<IReadOnlyList<LookupDto>> GetUnitsOfMeasureAsync(CancellationToken ct = default);
    Task<IReadOnlyList<LookupDto>> GetStockMovementTypesAsync(CancellationToken ct =
default);
    Task<IReadOnlyList<LookupDto>> GetActiveSuppliersAsync(CancellationToken ct = default);
    Task<IReadOnlyList<LookupDto>> GetStoreItemsAsync(string? searchTerm = null,
CancellationToken ct = default);
```

5. Infrastructure — DbContext changes

5.1 Six new DbSets

Excerpt — the six inventory DbSets

```
    public DbSet<ItemCategory> ItemCategories => Set<ItemCategory>();
    public DbSet<UnitOfMeasure> UnitsOfMeasure => Set<UnitOfMeasure>();
    public DbSet<StockMovementType> StockMovementTypes => Set<StockMovementType>();
    public DbSet<Supplier> Suppliers => Set<Supplier>();
    public DbSet<StoreItem> StoreItems => Set<StoreItem>();
    public DbSet<StockMovement> StockMovements => Set<StockMovement>();
```

5.2 ConfigureInventory

Excerpt — ConfigureInventory

```
    private static void ConfigureInventory(ModelBuilder builder)
    {
        ConfigureLookup<ItemCategory>(builder, "ItemCategories", extra: b =>
        {
            b.Property(c => c.Name).HasMaxLength(60).IsRequired();
            b.Property(c => c.Code).HasMaxLength(10).IsRequired();
            b.HasIndex(c => c.Name).IsUnique();
            b.HasIndex(c => c.Code).IsUnique();
        });

        ConfigureLookup<UnitOfMeasure>(builder, "UnitsOfMeasure", extra: b =>
        {
            b.Property(u => u.Name).HasMaxLength(40).IsRequired();
            b.Property(u => u.Code).HasMaxLength(10).IsRequired();
            b.HasIndex(u => u.Name).IsUnique();
            b.HasIndex(u => u.Code).IsUnique();
        });

        ConfigureLookup<StockMovementType>(builder, "StockMovementTypes", extra: b =>
        {
            b.Property(t => t.Name).HasMaxLength(40).IsRequired();
            b.Property(t => t.Code).HasMaxLength(20).IsRequired();
            b.HasIndex(t => t.Name).IsUnique();
            b.HasIndex(t => t.Code).IsUnique();
        });

        builder.Entity<Supplier>(b =>
        {
            b.ToTable("Suppliers");
            b.HasKey(s => s.Id);
            b.Property(s => s.Name).HasMaxLength(120).IsRequired();
            b.Property(s => s.ContactName).HasMaxLength(120);
            b.Property(s => s.Phone).HasMaxLength(30);
            b.Property(s => s.Email).HasMaxLength(256);
            b.Property(s => s.Address).HasMaxLength(300);
            b.Property(s => s.Notes).HasMaxLength(500);
            b.Property(s => s.CreatedBy).HasMaxLength(100);
```

```
            b.Property(s => s.ModifiedBy).HasMaxLength(100);
            b.Property(s => s.DeletedBy).HasMaxLength(100);

            b.HasIndex(s => s.Name);
            b.HasIndex(s => s.IsDeleted);
            b.HasQueryFilter(s => !s.IsDeleted);
        });

        builder.Entity<StoreItem>(b =>
        {
            b.ToTable("StoreItems");
            b.HasKey(i => i.Id);
            b.Property(i => i.Name).HasMaxLength(120).IsRequired();
            b.Property(i => i.Sku).HasMaxLength(60);
            b.Property(i => i.Description).HasMaxLength(500);
            b.Property(i => i.QuantityOnHand).HasPrecision(14, 3);
            b.Property(i => i.ReorderLevel).HasPrecision(14, 3);
            b.Property(i => i.LastUnitCost).HasPrecision(12, 2);
            b.Property(i => i.CreatedBy).HasMaxLength(100);
            b.Property(i => i.ModifiedBy).HasMaxLength(100);
            b.Property(i => i.DeletedBy).HasMaxLength(100);

            b.HasOne(i => i.ItemCategory).WithMany(c => c.Items)
                .HasForeignKey(i => i.ItemCategoryId)
                .OnDelete(DeleteBehavior.Restrict);

            b.HasOne(i => i.UnitOfMeasure).WithMany(u => u.Items)
                .HasForeignKey(i => i.UnitOfMeasureId)
                .OnDelete(DeleteBehavior.Restrict);

            b.HasIndex(i => i.Name);
            b.HasIndex(i => i.Sku).IsUnique()
                .HasFilter("[Sku] IS NOT NULL");
            b.HasIndex(i => i.IsDeleted);
            b.HasQueryFilter(i => !i.IsDeleted);
        });

        builder.Entity<StockMovement>(b =>
        {
            b.ToTable("StockMovements");
            b.HasKey(m => m.Id);
            b.Property(m => m.MovementNumber).HasMaxLength(40).IsRequired();
            b.Property(m => m.Quantity).HasPrecision(14, 3);
            b.Property(m => m.UnitCost).HasPrecision(12, 2);
            b.Property(m => m.TotalCost).HasPrecision(14, 2);
            b.Property(m => m.Reference).HasMaxLength(120);
            b.Property(m => m.Notes).HasMaxLength(300);
            b.Property(m => m.CreatedBy).HasMaxLength(100);
            b.Property(m => m.ModifiedBy).HasMaxLength(100);
            b.Property(m => m.DeletedBy).HasMaxLength(100);

            b.HasOne(m => m.StoreItem).WithMany(i => i.Movements)
                .HasForeignKey(m => m.StoreItemId)
                .OnDelete(DeleteBehavior.Restrict);
```

```
            b.HasOne(m => m.StockMovementType).WithMany(t => t.Movements)
                .HasForeignKey(m => m.StockMovementTypeId)
                .OnDelete(DeleteBehavior.Restrict);

            b.HasOne(m => m.ReceivedFromSupplier).WithMany(s => s.Purchases)
                .HasForeignKey(m => m.ReceivedFromSupplierId)
                .OnDelete(DeleteBehavior.SetNull);

            b.HasOne(m => m.IssuedToStudent).WithMany(s => s.StockIssuances)
                .HasForeignKey(m => m.IssuedToStudentId)
                .OnDelete(DeleteBehavior.SetNull);

            b.HasOne(m => m.IssuedToSchoolClass).WithMany(c => c.StockIssuances)
                .HasForeignKey(m => m.IssuedToSchoolClassId)
                .OnDelete(DeleteBehavior.SetNull);

            b.HasOne(m => m.IssuedToUser).WithMany()
                .HasForeignKey(m => m.IssuedToUserId)
                .OnDelete(DeleteBehavior.SetNull);

            b.HasOne(m => m.PerformedBy).WithMany()
                .HasForeignKey(m => m.PerformedById)
                .OnDelete(DeleteBehavior.SetNull);

            b.HasIndex(m => m.MovementNumber).IsUnique();
            b.HasIndex(m => new { m.StoreItemId, m.MovedOn });
            b.HasIndex(m => m.MovedOn);
            b.HasIndex(m => m.IsDeleted);
            b.HasQueryFilter(m => !m.IsDeleted);
        });
    }

    private static void ConfigureLookup<TEntity>(
```

Highlights:

- HasPrecision(14, 3) on Quantity and ReorderLevel; HasPrecision(12, 2) on UnitCost; HasPrecision(14, 2) on TotalCost.
- Filtered unique index on StoreItem.Sku — schools that don't use SKUs can leave the column null without colliding on the unique constraint.
- Restrict on StockMovement.StoreItemId / StockMovementTypeId — the schema-level constraint reinforces the service-layer guard that refuses to delete an item or type that has movements.
- SetNull on every optional counter-party (Supplier, Student, SchoolClass, IssuedToUser, PerformedBy) so soft-deleting any of those entities does not vaporise the audit history.
- Composite index on (StoreItemId, MovedOn) — speeds the per-item movement-history page.
- b.Ignore(i => i.StockValue) and b.Ignore(i => i.IsBelowReorder) on the DTO side — but the entity itself has no computed properties to ignore. Those are DTO conveniences.

6. Infrastructure — service implementations

6.1 SupplierService.cs

Listing — src/NaijaPrimeSchool.Infrastructure/Services/SupplierService.cs

```
using Microsoft.EntityFrameworkCore;
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Inventory;
using NaijaPrimeSchool.Application.Inventory.Dtos;
using NaijaPrimeSchool.Domain.Inventory;
using NaijaPrimeSchool.Infrastructure.Persistence;

namespace NaijaPrimeSchool.Infrastructure.Services;

public class SupplierService(ApplicationDbContext db) : ISupplierService
{
    private static IQueryable<SupplierDto> Project(IQueryable<Supplier> q) =>
        q.Select(s => new SupplierDto
        {
            Id = s.Id,
            Name = s.Name,
            ContactName = s.ContactName,
            Phone = s.Phone,
            Email = s.Email,
            Address = s.Address,
            Notes = s.Notes,
            IsActive = s.IsActive,
            PurchaseCount = s.Purchases.Count,
            TotalPurchased = s.Purchases.Sum(p => (decimal?)p.TotalCost) ?? 0m,
        });

    public async Task<IReadOnlyList<SupplierDto>> ListAsync(SupplierFilter filter,
CancellationToken ct = default)
    {
        var q = db.Suppliers.AsQueryable();
        if (filter.IsActive.HasValue) q = q.Where(s => s.IsActive ==
filter.IsActive.Value);
        if (!string.IsNullOrWhiteSpace(filter.Search))
        {
            var term = filter.Search.Trim().ToLower();
            q = q.Where(s =>
                s.Name.ToLower().Contains(term)
                || (s.ContactName != null && s.ContactName.ToLower().Contains(term))
                || (s.Phone != null && s.Phone.Contains(term))
                || (s.Email != null && s.Email.ToLower().Contains(term)));
        }
        return await Project(q.OrderBy(s => s.Name)).ToListAsync(ct);
    }

    public Task<SupplierDto?> GetByIdAsync(Guid id, CancellationToken ct = default) =>
        Project(db.Suppliers.Where(s => s.Id == id)).FirstOrDefaultAsync(ct);

    public async Task<OperationResult<Guid>> CreateAsync(CreateSupplierRequest request,
CancellationToken ct = default)
```

```
    {
        if (await db.Suppliers.AnyAsync(s => s.Name == request.Name, ct))
            return OperationResult<Guid>.Failure(
                $"A supplier named '{request.Name}' already exists.");

        var supplier = new Supplier
        {
            Name = request.Name.Trim(),
            ContactName = request.ContactName,
            Phone = request.Phone,
            Email = request.Email,
            Address = request.Address,
            Notes = request.Notes,
            IsActive = request.IsActive,
        };
        db.Suppliers.Add(supplier);
        await db.SaveChangesAsync(ct);
        return OperationResult<Guid>.Success(supplier.Id);
    }

    public async Task<OperationResult> UpdateAsync(UpdateSupplierRequest request,
CancellationToken ct = default)
    {
        var supplier = await db.Suppliers.FirstOrDefaultAsync(s => s.Id == request.Id, ct);
        if (supplier is null) return OperationResult.Failure("Supplier not found.");

        if (await db.Suppliers.AnyAsync(s => s.Name == request.Name && s.Id != request.Id,
ct))
            return OperationResult.Failure(
                $"A different supplier named '{request.Name}' already exists.");

        supplier.Name = request.Name.Trim();
        supplier.ContactName = request.ContactName;
        supplier.Phone = request.Phone;
        supplier.Email = request.Email;
        supplier.Address = request.Address;
        supplier.Notes = request.Notes;
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> SetActiveAsync(Guid id, bool isActive,
CancellationToken ct = default)
    {
        var supplier = await db.Suppliers.FirstOrDefaultAsync(s => s.Id == id, ct);
        if (supplier is null) return OperationResult.Failure("Supplier not found.");

        supplier.IsActive = isActive;
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct =
default)
    {
```

```
        var supplier = await db.Suppliers
            .Include(s => s.Purchases)
            .FirstOrDefaultAsync(s => s.Id == id, ct);
        if (supplier is null) return OperationResult.Failure("Supplier not found.");

        if (supplier.Purchases.Any())
            return OperationResult.Failure(
                "Cannot delete a supplier with purchase history. Deactivate instead.");

        db.Suppliers.Remove(supplier);
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }
}
```

6.2 StoreItemService.cs

StoreItemService owns catalog CRUD plus the create-with-opening-balance shortcut. When OpeningQuantity > 0 it stages a fresh StoreItem and an 'OPENING' StockMovement in the same SaveChanges, so a new item never lives in the system without its origin movement.

Listing — src/NaijaPrimeSchool.Infrastructure/Services/StoreItemService.cs

```
using Microsoft.EntityFrameworkCore;
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Inventory;
using NaijaPrimeSchool.Application.Inventory.Dtos;
using NaijaPrimeSchool.Domain.Inventory;
using NaijaPrimeSchool.Infrastructure.Persistence;

namespace NaijaPrimeSchool.Infrastructure.Services;

public class StoreItemService(ApplicationDbContext db) : IStoreItemService
{
    private static IQueryable<StoreItemDto> Project(IQueryable<StoreItem> q) =>
        q.Select(i => new StoreItemDto
        {
            Id = i.Id,
            Name = i.Name,
            Sku = i.Sku,
            Description = i.Description,
            ItemCategoryId = i.ItemCategoryId,
            ItemCategoryName = i.ItemCategory!.Name,
            UnitOfMeasureId = i.UnitOfMeasureId,
            UnitOfMeasureName = i.UnitOfMeasure!.Name,
            UnitOfMeasureCode = i.UnitOfMeasure!.Code,
            QuantityOnHand = i.QuantityOnHand,
            ReorderLevel = i.ReorderLevel,
            LastUnitCost = i.LastUnitCost,
            IsActive = i.IsActive,
        });

    public async Task<IReadOnlyList<StoreItemDto>> ListAsync(StoreItemFilter filter,
CancellationToken ct = default)
```

```
    {
        var q = db.StoreItems.AsQueryable();
        if (filter.ItemCategoryId.HasValue) q = q.Where(i => i.ItemCategoryId ==
filter.ItemCategoryId.Value);
        if (filter.IsActive.HasValue) q = q.Where(i => i.IsActive ==
filter.IsActive.Value);
        if (filter.OnlyBelowReorder == true) q = q.Where(i => i.QuantityOnHand <=
i.ReorderLevel);
        if (!string.IsNullOrWhiteSpace(filter.Search))
        {
            var term = filter.Search.Trim().ToLower();
            q = q.Where(i =>
                i.Name.ToLower().Contains(term)
                || (i.Sku != null && i.Sku.ToLower().Contains(term)));
        }
        return await Project(q.OrderBy(i => i.Name)).ToListAsync(ct);
    }

    public async Task<StoreItemDetailDto?> GetByIdAsync(Guid id, CancellationToken ct =
default)
    {
        var item = await Project(db.StoreItems.Where(i => i.Id ==
id)).FirstOrDefaultAsync(ct);
        if (item is null) return null;

        var movements = await db.StockMovements
            .Where(m => m.StoreItemId == id)
            .OrderByDescending(m => m.MovedOn)
            .ThenByDescending(m => m.MovementNumber)
            .Take(100)
            .Select(m => new StockMovementDto
            {
                Id = m.Id,
                StoreItemId = m.StoreItemId,
                StoreItemName = item.Name,
                StoreItemSku = item.Sku,
                UnitOfMeasureCode = item.UnitOfMeasureCode,
                StockMovementTypeId = m.StockMovementTypeId,
                StockMovementTypeName = m.StockMovementType!.Name,
                StockMovementTypeCode = m.StockMovementType!.Code,
                Direction = m.StockMovementType!.Direction,
                MovementNumber = m.MovementNumber,
                MovedOn = m.MovedOn,
                Quantity = m.Quantity,
                UnitCost = m.UnitCost,
                TotalCost = m.TotalCost,
                Reference = m.Reference,
                Notes = m.Notes,
                ReceivedFromSupplierId = m.ReceivedFromSupplierId,
                ReceivedFromSupplierName = m.ReceivedFromSupplier == null
                    ? null : m.ReceivedFromSupplier.Name,
                IssuedToStudentId = m.IssuedToStudentId,
                IssuedToStudentName = m.IssuedToStudent == null
                    ? null : (m.IssuedToStudent!.FirstName + " " +
m.IssuedToStudent!.LastName).Trim(),
```

```
                IssuedToStudentAdmissionNumber = m.IssuedToStudent == null
                    ? null : m.IssuedToStudent!.AdmissionNumber,
                IssuedToStudentPhotoUrl = m.IssuedToStudent == null
                    ? null : m.IssuedToStudent!.PhotoUrl,
                IssuedToStudentFirstName = m.IssuedToStudent == null
                    ? null : m.IssuedToStudent!.FirstName,
                IssuedToStudentLastName = m.IssuedToStudent == null
                    ? null : m.IssuedToStudent!.LastName,
                IssuedToSchoolClassId = m.IssuedToSchoolClassId,
                IssuedToSchoolClassName = m.IssuedToSchoolClass == null
                    ? null : m.IssuedToSchoolClass!.Name,
                IssuedToUserId = m.IssuedToUserId,
                IssuedToUserName = m.IssuedToUser == null
                    ? null : (m.IssuedToUser!.FirstName + " " +
m.IssuedToUser!.LastName).Trim(),
                PerformedById = m.PerformedById,
                PerformedByName = m.PerformedBy == null
                    ? null : (m.PerformedBy!.FirstName + " " +
m.PerformedBy!.LastName).Trim(),
            })
            .ToListAsync(ct);

        return new StoreItemDetailDto { Item = item, Movements = movements };
    }

    public async Task<OperationResult<Guid>> CreateAsync(CreateStoreItemRequest request,
CancellationToken ct = default)
    {
        if (!await db.ItemCategories.AnyAsync(c => c.Id == request.ItemCategoryId, ct))
            return OperationResult<Guid>.Failure("Item category not found.");

        if (!await db.UnitsOfMeasure.AnyAsync(u => u.Id == request.UnitOfMeasureId, ct))
            return OperationResult<Guid>.Failure("Unit of measure not found.");

        if (!string.IsNullOrWhiteSpace(request.Sku)
            && await db.StoreItems.AnyAsync(i => i.Sku == request.Sku, ct))
            return OperationResult<Guid>.Failure(
                $"An item with SKU '{request.Sku}' already exists.");

        var item = new StoreItem
        {
            Name = request.Name.Trim(),
            Sku = string.IsNullOrWhiteSpace(request.Sku) ? null : request.Sku.Trim(),
            Description = request.Description,
            ItemCategoryId = request.ItemCategoryId,
            UnitOfMeasureId = request.UnitOfMeasureId,
            ReorderLevel = request.ReorderLevel,
            QuantityOnHand = 0m,
            LastUnitCost = request.OpeningUnitCost,
            IsActive = request.IsActive,
        };
        db.StoreItems.Add(item);

        if (request.OpeningQuantity > 0m)
        {
```

```
            var openingTypeId = await db.StockMovementTypes
                .Where(t => t.Code == "OPENING")
                .Select(t => t.Id)
                .FirstOrDefaultAsync(ct);
            if (openingTypeId == Guid.Empty)
                return OperationResult<Guid>.Failure(
                    "Stock movement types are not seeded.");

            var movementNumber = await NextMovementNumberAsync(request.OpeningUnitCost >
0 ? DateTime.UtcNow.Year : DateTime.UtcNow.Year, ct);

            db.StockMovements.Add(new StockMovement
            {
                StoreItem = item,
                StockMovementTypeId = openingTypeId,
                MovementNumber = movementNumber,
                MovedOn = DateOnly.FromDateTime(DateTime.UtcNow),
                Quantity = request.OpeningQuantity,
                UnitCost = request.OpeningUnitCost,
                TotalCost = request.OpeningQuantity * request.OpeningUnitCost,
                Reference = "Opening balance",
            });

            item.QuantityOnHand = request.OpeningQuantity;
        }

        await db.SaveChangesAsync(ct);
        return OperationResult<Guid>.Success(item.Id);
    }

    public async Task<OperationResult> UpdateAsync(UpdateStoreItemRequest request,
CancellationToken ct = default)
    {
        var item = await db.StoreItems.FirstOrDefaultAsync(i => i.Id == request.Id, ct);
        if (item is null) return OperationResult.Failure("Item not found.");

        if (!await db.ItemCategories.AnyAsync(c => c.Id == request.ItemCategoryId, ct))
            return OperationResult.Failure("Item category not found.");

        if (!await db.UnitsOfMeasure.AnyAsync(u => u.Id == request.UnitOfMeasureId, ct))
            return OperationResult.Failure("Unit of measure not found.");

        if (!string.IsNullOrWhiteSpace(request.Sku)
            && await db.StoreItems.AnyAsync(i => i.Sku == request.Sku && i.Id !=
request.Id, ct))
            return OperationResult.Failure(
                $"An item with SKU '{request.Sku}' already exists.");

        item.Name = request.Name.Trim();
        item.Sku = string.IsNullOrWhiteSpace(request.Sku) ? null : request.Sku.Trim();
        item.Description = request.Description;
        item.ItemCategoryId = request.ItemCategoryId;
        item.UnitOfMeasureId = request.UnitOfMeasureId;
        item.ReorderLevel = request.ReorderLevel;
        await db.SaveChangesAsync(ct);
```

```
        return OperationResult.Success();
    }

    public async Task<OperationResult> SetActiveAsync(Guid id, bool isActive,
CancellationToken ct = default)
    {
        var item = await db.StoreItems.FirstOrDefaultAsync(i => i.Id == id, ct);
        if (item is null) return OperationResult.Failure("Item not found.");

        item.IsActive = isActive;
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct =
default)
    {
        var item = await db.StoreItems
            .Include(i => i.Movements)
            .FirstOrDefaultAsync(i => i.Id == id, ct);
        if (item is null) return OperationResult.Failure("Item not found.");

        if (item.Movements.Any())
            return OperationResult.Failure(
                "Cannot delete an item with movement history. Deactivate instead.");

        db.StoreItems.Remove(item);
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    private async Task<string> NextMovementNumberAsync(int year, CancellationToken ct)
    {
        var prefix = $"NPS/STK/{year}/";
        var existing = await db.StockMovements
            .Where(m => m.MovementNumber.StartsWith(prefix))
            .Select(m => m.MovementNumber)
            .ToListAsync(ct);
        var max = 0;
        foreach (var n in existing)
        {
            if (int.TryParse(n[prefix.Length..], out var seq) && seq > max) max = seq;
        }
        return prefix + (max + 1).ToString("D4");
    }
}
```

6.3 StockMovementService.cs

The audit-trail service. RecordAsync validates the type, checks the at-most-one-recipient rule, refuses outbound movements that would drive the on-hand quantity negative, computes the next sequential movement number, persists the row and updates StoreItem.QuantityOnHand and (for inbound with a unit

cost) LastUnitCost in the same SaveChanges. SoftDeleteAsync reverses the contribution before removing the row.

Listing — src/NaijaPrimeSchool.Infrastructure/Services/StockMovementService.cs

```
using Microsoft.EntityFrameworkCore;
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Inventory;
using NaijaPrimeSchool.Application.Inventory.Dtos;
using NaijaPrimeSchool.Domain.Inventory;
using NaijaPrimeSchool.Infrastructure.Persistence;

namespace NaijaPrimeSchool.Infrastructure.Services;

public class StockMovementService(ApplicationDbContext db) : IStockMovementService
{
    private static IQueryable<StockMovementDto> Project(IQueryable<StockMovement> q) =>
        q.Select(m => new StockMovementDto
        {
            Id = m.Id,
            StoreItemId = m.StoreItemId,
            StoreItemName = m.StoreItem!.Name,
            StoreItemSku = m.StoreItem!.Sku,
            UnitOfMeasureCode = m.StoreItem!.UnitOfMeasure!.Code,
            StockMovementTypeId = m.StockMovementTypeId,
            StockMovementTypeName = m.StockMovementType!.Name,
            StockMovementTypeCode = m.StockMovementType!.Code,
            Direction = m.StockMovementType!.Direction,
            MovementNumber = m.MovementNumber,
            MovedOn = m.MovedOn,
            Quantity = m.Quantity,
            UnitCost = m.UnitCost,
            TotalCost = m.TotalCost,
            Reference = m.Reference,
            Notes = m.Notes,
            ReceivedFromSupplierId = m.ReceivedFromSupplierId,
            ReceivedFromSupplierName = m.ReceivedFromSupplier == null
                ? null : m.ReceivedFromSupplier.Name,
            IssuedToStudentId = m.IssuedToStudentId,
            IssuedToStudentName = m.IssuedToStudent == null
                ? null : (m.IssuedToStudent!.FirstName + " " +
m.IssuedToStudent!.LastName).Trim(),
            IssuedToStudentAdmissionNumber = m.IssuedToStudent == null
                ? null : m.IssuedToStudent!.AdmissionNumber,
            IssuedToStudentPhotoUrl = m.IssuedToStudent == null
                ? null : m.IssuedToStudent!.PhotoUrl,
            IssuedToStudentFirstName = m.IssuedToStudent == null
                ? null : m.IssuedToStudent!.FirstName,
            IssuedToStudentLastName = m.IssuedToStudent == null
                ? null : m.IssuedToStudent!.LastName,
            IssuedToSchoolClassId = m.IssuedToSchoolClassId,
            IssuedToSchoolClassName = m.IssuedToSchoolClass == null
                ? null : m.IssuedToSchoolClass!.Name,
            IssuedToUserId = m.IssuedToUserId,
            IssuedToUserName = m.IssuedToUser == null
```

```
                ? null : (m.IssuedToUser!.FirstName + " " +
m.IssuedToUser!.LastName).Trim(),
            PerformedById = m.PerformedById,
            PerformedByName = m.PerformedBy == null
                ? null : (m.PerformedBy!.FirstName + " " + m.PerformedBy!.LastName).Trim(),
        });

    public async Task<IReadOnlyList<StockMovementDto>> ListAsync(StockMovementFilter
filter, CancellationToken ct = default)
    {
        var q = db.StockMovements.AsQueryable();
        if (filter.StoreItemId.HasValue) q = q.Where(m => m.StoreItemId ==
filter.StoreItemId.Value);
        if (filter.StockMovementTypeId.HasValue) q = q.Where(m => m.StockMovementTypeId ==
filter.StockMovementTypeId.Value);
        if (filter.SupplierId.HasValue) q = q.Where(m => m.ReceivedFromSupplierId ==
filter.SupplierId.Value);
        if (filter.Direction.HasValue) q = q.Where(m => m.StockMovementType!.Direction ==
filter.Direction.Value);
        if (filter.FromDate.HasValue) q = q.Where(m => m.MovedOn >= filter.FromDate.Value);
        if (filter.ToDate.HasValue) q = q.Where(m => m.MovedOn <= filter.ToDate.Value);
        if (!string.IsNullOrWhiteSpace(filter.Search))
        {
            var term = filter.Search.Trim().ToLower();
            q = q.Where(m =>
                m.MovementNumber.ToLower().Contains(term)
                || m.StoreItem!.Name.ToLower().Contains(term)
                || (m.StoreItem!.Sku != null && m.StoreItem!.Sku.ToLower().Contains(term))
                || (m.Reference != null && m.Reference.ToLower().Contains(term)));
        }

        return await Project(q.OrderByDescending(m => m.MovedOn).ThenByDescending(m =>
m.MovementNumber))
            .ToListAsync(ct);
    }

    public Task<StockMovementDto?> GetByIdAsync(Guid id, CancellationToken ct = default) =>
        Project(db.StockMovements.Where(m => m.Id == id)).FirstOrDefaultAsync(ct);

    public async Task<OperationResult<Guid>> RecordAsync(RecordStockMovementRequest
request, CancellationToken ct = default)
    {
        if (request.Quantity <= 0m)
            return OperationResult<Guid>.Failure("Quantity must be greater than zero.");

        var item = await db.StoreItems.FirstOrDefaultAsync(i => i.Id ==
request.StoreItemId, ct);
        if (item is null) return OperationResult<Guid>.Failure("Store item not found.");

        var type = await db.StockMovementTypes
            .FirstOrDefaultAsync(t => t.Id == request.StockMovementTypeId, ct);
        if (type is null) return OperationResult<Guid>.Failure("Stock movement type not
found.");

        if (type.Direction == -1 && request.Quantity > item.QuantityOnHand)
```

```
            return OperationResult<Guid>.Failure(
                $"Cannot remove {request.Quantity:N3} {item.QuantityOnHand:N3} are on
hand.");

        // Counter-party validation: at most one IssuedTo* and the IsValidForType
        // pairing (purchases name a supplier, issues name a recipient).
        var counterPartyCount = 0;
        if (request.IssuedToStudentId.HasValue) counterPartyCount++;
        if (request.IssuedToSchoolClassId.HasValue) counterPartyCount++;
        if (request.IssuedToUserId.HasValue) counterPartyCount++;
        if (counterPartyCount > 1)
            return OperationResult<Guid>.Failure(
                "A movement can be issued to at most one recipient (pupil, class, or staff
member).");

        if (request.ReceivedFromSupplierId.HasValue
            && !await db.Suppliers.AnyAsync(s => s.Id ==
request.ReceivedFromSupplierId.Value, ct))
            return OperationResult<Guid>.Failure("Selected supplier not found.");

        if (request.IssuedToStudentId.HasValue
            && !await db.Students.AnyAsync(s => s.Id == request.IssuedToStudentId.Value,
ct))
            return OperationResult<Guid>.Failure("Selected pupil not found.");

        if (request.IssuedToSchoolClassId.HasValue
            && !await db.SchoolClasses.AnyAsync(c => c.Id ==
request.IssuedToSchoolClassId.Value, ct))
            return OperationResult<Guid>.Failure("Selected class not found.");

        // Receipt-style sequential numbering, year-prefixed.
        var year = request.MovedOn.Year;
        var prefix = $"NPS/STK/{year}/";
        var existing = await db.StockMovements
            .Where(m => m.MovementNumber.StartsWith(prefix))
            .Select(m => m.MovementNumber)
            .ToListAsync(ct);
        var max = 0;
        foreach (var n in existing)
        {
            if (int.TryParse(n[prefix.Length..], out var seq) && seq > max) max = seq;
        }
        var movementNumber = prefix + (max + 1).ToString("D4");

        var movement = new StockMovement
        {
            StoreItemId = request.StoreItemId,
            StockMovementTypeId = request.StockMovementTypeId,
            MovementNumber = movementNumber,
            MovedOn = request.MovedOn,
            Quantity = request.Quantity,
            UnitCost = request.UnitCost,
            TotalCost = request.Quantity * request.UnitCost,
            Reference = request.Reference,
            Notes = request.Notes,
```

```
            ReceivedFromSupplierId = request.ReceivedFromSupplierId,
            IssuedToStudentId = request.IssuedToStudentId,
            IssuedToSchoolClassId = request.IssuedToSchoolClassId,
            IssuedToUserId = request.IssuedToUserId,
            PerformedById = request.PerformedById,
        };
        db.StockMovements.Add(movement);

        // Update the running balance on the item using Direction.
        item.QuantityOnHand += type.Direction * request.Quantity;
        if (type.Direction == +1 && request.UnitCost > 0m)
        {
            // Latest known unit cost feeds inventory valuation in the dashboard.
            item.LastUnitCost = request.UnitCost;
        }

        await db.SaveChangesAsync(ct);
        return OperationResult<Guid>.Success(movement.Id);
    }

    public async Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct =
default)
    {
        var movement = await db.StockMovements
            .Include(m => m.StockMovementType)
            .FirstOrDefaultAsync(m => m.Id == id, ct);
        if (movement is null) return OperationResult.Failure("Movement not found.");

        // Reverse the running balance on the item so the audit history can be
        // hidden without leaving a phantom quantity on the catalog.
        var item = await db.StoreItems.FirstOrDefaultAsync(i => i.Id ==
movement.StoreItemId, ct);
        if (item is not null)
        {
            item.QuantityOnHand -= movement.StockMovementType!.Direction *
movement.Quantity;
        }

        db.StockMovements.Remove(movement);
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<StoreSummaryDto> GetStoreSummaryAsync(CancellationToken ct = default)
    {
        var items = await db.StoreItems
            .Select(i => new
            {
                i.Id,
                i.Name,
                i.Sku,
                i.IsActive,
                i.QuantityOnHand,
                i.ReorderLevel,
                i.LastUnitCost,
```

```
                i.ItemCategoryId,
                CategoryName = i.ItemCategory!.Name,
                UnitName = i.UnitOfMeasure!.Name,
                UnitCode = i.UnitOfMeasure!.Code,
            })
            .ToListAsync(ct);

        var monthStart = new DateOnly(DateTime.UtcNow.Year, DateTime.UtcNow.Month, 1);
        var monthEnd = monthStart.AddMonths(1);

        var movementRows = await db.StockMovements
            .Where(m => m.MovedOn >= monthStart && m.MovedOn < monthEnd)
            .Select(m => new { m.StockMovementType!.Direction })
            .ToListAsync(ct);

        var recent = await Project(db.StockMovements.OrderByDescending(m => m.MovedOn)
                                                    .ThenByDescending(m =>
m.MovementNumber)
                                                    .Take(20))
            .ToListAsync(ct);

        var summary = new StoreSummaryDto
        {
            TotalItems = items.Count,
            ActiveItems = items.Count(i => i.IsActive),
            ItemsBelowReorder = items.Count(i => i.IsActive && i.QuantityOnHand <=
i.ReorderLevel),
            StockValue = items.Sum(i => i.QuantityOnHand * i.LastUnitCost),
            MovementsThisMonth = movementRows.Count,
            InboundThisMonth = movementRows.Count(m => m.Direction == +1),
            OutboundThisMonth = movementRows.Count(m => m.Direction == -1),
            ByCategory = items
                .GroupBy(i => new { i.ItemCategoryId, i.CategoryName })
                .Select(g => new CategoryStockDto
                {
                    ItemCategoryId = g.Key.ItemCategoryId,
                    ItemCategoryName = g.Key.CategoryName,
                    ItemCount = g.Count(),
                    StockValue = g.Sum(i => i.QuantityOnHand * i.LastUnitCost),
                })
                .OrderByDescending(b => b.StockValue)
                .ToList(),
            LowStockItems = items
                .Where(i => i.IsActive && i.QuantityOnHand <= i.ReorderLevel)
                .OrderBy(i => i.QuantityOnHand)
                .Take(15)
                .Select(i => new StoreItemDto
                {
                    Id = i.Id,
                    Name = i.Name,
                    Sku = i.Sku,
                    ItemCategoryId = i.ItemCategoryId,
                    ItemCategoryName = i.CategoryName,
                    UnitOfMeasureName = i.UnitName,
                    UnitOfMeasureCode = i.UnitCode,
```

```
                    QuantityOnHand = i.QuantityOnHand,
                    ReorderLevel = i.ReorderLevel,
                    LastUnitCost = i.LastUnitCost,
                    IsActive = i.IsActive,
                })
                .ToList(),
            RecentMovements = recent.ToList(),
        };

        return summary;
    }
}
```

6.4 DI registration

Excerpt — DependencyInjection.cs (sprint 7 additions)

```
        services.AddScoped<ISupplierService, SupplierService>();
        services.AddScoped<IStoreItemService, StoreItemService>();
        services.AddScoped<IStockMovementService, StockMovementService>();

        return services;
```

6.5 LookupService — five new methods

Excerpt — LookupService.cs (sprint 7 additions)

```
    public async Task<IReadOnlyList<LookupDto>> GetItemCategoriesAsync(CancellationToken ct
= default) =>
        await db.ItemCategories
            .OrderBy(c => c.DisplayOrder)
            .Select(c => new LookupDto { Id = c.Id, Name = c.Name, Code = c.Code })
            .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetUnitsOfMeasureAsync(CancellationToken ct
= default) =>
        await db.UnitsOfMeasure
            .OrderBy(u => u.DisplayOrder)
            .Select(u => new LookupDto { Id = u.Id, Name = u.Name, Code = u.Code })
            .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>>
GetStockMovementTypesAsync(CancellationToken ct = default) =>
        await db.StockMovementTypes
            .OrderBy(t => t.DisplayOrder)
            .Select(t => new LookupDto { Id = t.Id, Name = t.Name, Code = t.Code })
            .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetActiveSuppliersAsync(CancellationToken
ct = default) =>
        await db.Suppliers
            .Where(s => s.IsActive)
            .OrderBy(s => s.Name)
            .Select(s => new LookupDto { Id = s.Id, Name = s.Name })
```

```
            .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetStoreItemsAsync(string? searchTerm =
null, CancellationToken ct = default)
    {
        var q = db.StoreItems.Where(i => i.IsActive);
        if (!string.IsNullOrWhiteSpace(searchTerm))
        {
            var term = searchTerm.Trim().ToLower();
            q = q.Where(i =>
                i.Name.ToLower().Contains(term)
                || (i.Sku != null && i.Sku.ToLower().Contains(term)));
        }
        return await q
            .OrderBy(i => i.Name)
            .Take(50)
            .Select(i => new LookupDto
            {
                Id = i.Id,
                Name = i.Name,
                Code = i.Sku,
            })
            .ToListAsync(ct);
    }
}
```

7. The EF Core migration

```
dotnet ef migrations add StoreAndInventory \
  --project src/NaijaPrimeSchool.Infrastructure \
  --startup-project src/NaijaPrimeSchool.Web \
  --output-dir Persistence/Migrations
```

Excerpt — Up() of 20260516054730_StoreAndInventory.cs

```
        protected override void Up(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.CreateTable(
                name: "ItemCategories",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    Name = table.Column<string>(type: "nvarchar(60)", maxLength: 60,
nullable: false),
                    Code = table.Column<string>(type: "nvarchar(10)", maxLength: 10,
nullable: false),
                    DisplayOrder = table.Column<int>(type: "int", nullable: false),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_ItemCategories", x => x.Id);
                });

            migrationBuilder.CreateTable(
                name: "StockMovementTypes",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    Name = table.Column<string>(type: "nvarchar(40)", maxLength: 40,
nullable: false),
                    Code = table.Column<string>(type: "nvarchar(20)", maxLength: 20,
nullable: false),
                    Direction = table.Column<int>(type: "int", nullable: false),
                    DisplayOrder = table.Column<int>(type: "int", nullable: false),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
```

```
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_StockMovementTypes", x => x.Id);
                });

            migrationBuilder.CreateTable(
                name: "Suppliers",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    Name = table.Column<string>(type: "nvarchar(120)", maxLength: 120,
nullable: false),
                    ContactName = table.Column<string>(type: "nvarchar(120)", maxLength:
120, nullable: true),
                    Phone = table.Column<string>(type: "nvarchar(30)", maxLength: 30,
nullable: true),
                    Email = table.Column<string>(type: "nvarchar(256)", maxLength: 256,
nullable: true),
                    Address = table.Column<string>(type: "nvarchar(300)", maxLength: 300,
nullable: true),
                    Notes = table.Column<string>(type: "nvarchar(500)", maxLength: 500,
nullable: true),
                    IsActive = table.Column<bool>(type: "bit", nullable: false),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_Suppliers", x => x.Id);
                });

            migrationBuilder.CreateTable(
                name: "UnitsOfMeasure",
                columns: table => new
```

```
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    Name = table.Column<string>(type: "nvarchar(40)", maxLength: 40,
nullable: false),
                    Code = table.Column<string>(type: "nvarchar(10)", maxLength: 10,
nullable: false),
                    DisplayOrder = table.Column<int>(type: "int", nullable: false),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_UnitsOfMeasure", x => x.Id);
                });

            migrationBuilder.CreateTable(
                name: "StoreItems",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    Name = table.Column<string>(type: "nvarchar(120)", maxLength: 120,
nullable: false),
                    Sku = table.Column<string>(type: "nvarchar(60)", maxLength: 60,
nullable: true),
                    Description = table.Column<string>(type: "nvarchar(500)", maxLength:
500, nullable: true),
                    ItemCategoryId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
                    UnitOfMeasureId = table.Column<Guid>(type: "uniqueidentifier",
nullable: false),
                    QuantityOnHand = table.Column<decimal>(type: "decimal(14,3)",
precision: 14, scale: 3, nullable: false),
                    ReorderLevel = table.Column<decimal>(type: "decimal(14,3)", precision:
14, scale: 3, nullable: false),
                    LastUnitCost = table.Column<decimal>(type: "decimal(12,2)", precision:
12, scale: 2, nullable: false),
                    IsActive = table.Column<bool>(type: "bit", nullable: false),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
```

```
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_StoreItems", x => x.Id);
                    table.ForeignKey(
                        name: "FK_StoreItems_ItemCategories_ItemCategoryId",
                        column: x => x.ItemCategoryId,
                        principalTable: "ItemCategories",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                    table.ForeignKey(
                        name: "FK_StoreItems_UnitsOfMeasure_UnitOfMeasureId",
                        column: x => x.UnitOfMeasureId,
                        principalTable: "UnitsOfMeasure",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                });

            migrationBuilder.CreateTable(
                name: "StockMovements",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    StoreItemId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
                    StockMovementTypeId = table.Column<Guid>(type: "uniqueidentifier",
nullable: false),
                    MovementNumber = table.Column<string>(type: "nvarchar(40)", maxLength:
40, nullable: false),
                    MovedOn = table.Column<DateOnly>(type: "date", nullable: false),
                    Quantity = table.Column<decimal>(type: "decimal(14,3)", precision: 14,
scale: 3, nullable: false),
                    UnitCost = table.Column<decimal>(type: "decimal(12,2)", precision: 12,
scale: 2, nullable: false),
                    TotalCost = table.Column<decimal>(type: "decimal(14,2)", precision: 14,
scale: 2, nullable: false),
                    Reference = table.Column<string>(type: "nvarchar(120)", maxLength: 120,
nullable: true),
                    Notes = table.Column<string>(type: "nvarchar(300)", maxLength: 300,
nullable: true),
                    ReceivedFromSupplierId = table.Column<Guid>(type: "uniqueidentifier",
nullable: true),
                    IssuedToStudentId = table.Column<Guid>(type: "uniqueidentifier",
nullable: true),
                    IssuedToSchoolClassId = table.Column<Guid>(type: "uniqueidentifier",
nullable: true),
                    IssuedToUserId = table.Column<Guid>(type: "uniqueidentifier", nullable:
true),
                    PerformedById = table.Column<Guid>(type: "uniqueidentifier", nullable:
```

```
true),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_StockMovements", x => x.Id);
                    table.ForeignKey(
                        name: "FK_StockMovements_SchoolClasses_IssuedToSchoolClassId",
                        column: x => x.IssuedToSchoolClassId,
                        principalTable: "SchoolClasses",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.SetNull);
                    table.ForeignKey(
                        name: "FK_StockMovements_StockMovementTypes_StockMovementTypeId",
                        column: x => x.StockMovementTypeId,
                        principalTable: "StockMovementTypes",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                    table.ForeignKey(
                        name: "FK_StockMovements_StoreItems_StoreItemId",
                        column: x => x.StoreItemId,
                        principalTable: "StoreItems",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                    table.ForeignKey(
                        name: "FK_StockMovements_Students_IssuedToStudentId",
                        column: x => x.IssuedToStudentId,
                        principalTable: "Students",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.SetNull);
                    table.ForeignKey(
                        name: "FK_StockMovements_Suppliers_ReceivedFromSupplierId",
                        column: x => x.ReceivedFromSupplierId,
                        principalTable: "Suppliers",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.SetNull);
                    table.ForeignKey(
                        name: "FK_StockMovements_Users_IssuedToUserId",
                        column: x => x.IssuedToUserId,
                        principalTable: "Users",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.SetNull);
                    table.ForeignKey(
```

```
                name: "FK_StockMovements_Users_PerformedById",
                column: x => x.PerformedById,
                principalTable: "Users",
                principalColumn: "Id",
                onDelete: ReferentialAction.SetNull);
        });

    migrationBuilder.CreateIndex(
        name: "IX_ItemCategories_Code",
        table: "ItemCategories",
        column: "Code",
        unique: true);

    migrationBuilder.CreateIndex(
        name: "IX_ItemCategories_Name",
        table: "ItemCategories",
        column: "Name",
        unique: true);

    migrationBuilder.CreateIndex(
        name: "IX_StockMovements_IsDeleted",
        table: "StockMovements",
        column: "IsDeleted");

    migrationBuilder.CreateIndex(
        name: "IX_StockMovements_IssuedToSchoolClassId",
        table: "StockMovements",
        column: "IssuedToSchoolClassId");

    migrationBuilder.CreateIndex(
        name: "IX_StockMovements_IssuedToStudentId",
        table: "StockMovements",
        column: "IssuedToStudentId");

    migrationBuilder.CreateIndex(
        name: "IX_StockMovements_IssuedToUserId",
        table: "StockMovements",
        column: "IssuedToUserId");

    migrationBuilder.CreateIndex(
        name: "IX_StockMovements_MovedOn",
        table: "StockMovements",
        column: "MovedOn");

    migrationBuilder.CreateIndex(
        name: "IX_StockMovements_MovementNumber",
        table: "StockMovements",
        column: "MovementNumber",
        unique: true);

    migrationBuilder.CreateIndex(
        name: "IX_StockMovements_PerformedById",
        table: "StockMovements",
        column: "PerformedById");
```

```
            migrationBuilder.CreateIndex(
                name: "IX_StockMovements_ReceivedFromSupplierId",
                table: "StockMovements",
                column: "ReceivedFromSupplierId");

            migrationBuilder.CreateIndex(
                name: "IX_StockMovements_StockMovementTypeId",
                table: "StockMovements",
                column: "StockMovementTypeId");

            migrationBuilder.CreateIndex(
                name: "IX_StockMovements_StoreItemId_MovedOn",
                table: "StockMovements",
                columns: new[] { "StoreItemId", "MovedOn" });

            migrationBuilder.CreateIndex(
                name: "IX_StockMovementTypes_Code",
                table: "StockMovementTypes",
                column: "Code",
                unique: true);

            migrationBuilder.CreateIndex(
                name: "IX_StockMovementTypes_Name",
                table: "StockMovementTypes",
                column: "Name",
                unique: true);

            migrationBuilder.CreateIndex(
                name: "IX_StoreItems_IsDeleted",
                table: "StoreItems",
                column: "IsDeleted");

            migrationBuilder.CreateIndex(
                name: "IX_StoreItems_ItemCategoryId",
                table: "StoreItems",
                column: "ItemCategoryId");

            migrationBuilder.CreateIndex(
                name: "IX_StoreItems_Name",
                table: "StoreItems",
                column: "Name");

            migrationBuilder.CreateIndex(
                name: "IX_StoreItems_Sku",
                table: "StoreItems",
                column: "Sku",
                unique: true,
                filter: "[Sku] IS NOT NULL");

            migrationBuilder.CreateIndex(
                name: "IX_StoreItems_UnitOfMeasureId",
                table: "StoreItems",
                column: "UnitOfMeasureId");

            migrationBuilder.CreateIndex(
```

```
                name: "IX_Suppliers_IsDeleted",
                table: "Suppliers",
                column: "IsDeleted");

            migrationBuilder.CreateIndex(
                name: "IX_Suppliers_Name",
                table: "Suppliers",
                column: "Name");

            migrationBuilder.CreateIndex(
                name: "IX_UnitsOfMeasure_Code",
                table: "UnitsOfMeasure",
                column: "Code",
                unique: true);

            migrationBuilder.CreateIndex(
                name: "IX_UnitsOfMeasure_Name",
                table: "UnitsOfMeasure",
                column: "Name",
                unique: true);
        }

        /// <inheritdoc />
        protected override void Down(MigrationBuilder migrationBuilder)
```

8. Seeding the inventory lookups

Excerpt — SeedInventoryLookupsAsync

```
    private static async Task SeedInventoryLookupsAsync(ApplicationDbContext db,
CancellationToken ct)
    {
        if (!await db.ItemCategories.IgnoreQueryFilters().AnyAsync(ct))
        {
            (string Name, string Code)[] categories =
            [
                ("Books",            "BOOK"),
                ("Uniforms",         "UNIF"),
                ("Stationery",       "STAT"),
                ("Sports Equipment", "SPRT"),
                ("Cleaning Supplies","CLN"),
                ("Food",             "FOOD"),
                ("Furniture",        "FURN"),
                ("ICT Equipment",    "ICT"),
                ("Medical Supplies", "MED"),
                ("Other",            "OTH"),
            ];
            for (var i = 0; i < categories.Length; i++)
            {
                db.ItemCategories.Add(new ItemCategory
                {
                    Name = categories[i].Name,
                    Code = categories[i].Code,
                    DisplayOrder = i + 1,
                });
            }
        }

        if (!await db.UnitsOfMeasure.IgnoreQueryFilters().AnyAsync(ct))
        {
            (string Name, string Code)[] units =
            [
                ("Each",     "EA"),
                ("Piece",    "PC"),
                ("Pack",     "PK"),
                ("Box",      "BOX"),
                ("Carton",   "CTN"),
                ("Set",      "SET"),
                ("Bag",      "BAG"),
                ("Kilogram", "KG"),
                ("Litre",    "L"),
                ("Metre",    "M"),
            ];
            for (var i = 0; i < units.Length; i++)
            {
                db.UnitsOfMeasure.Add(new UnitOfMeasure
                {
                    Name = units[i].Name,
                    Code = units[i].Code,
                    DisplayOrder = i + 1,
```

```
                });
            }
        }

        if (!await db.StockMovementTypes.IgnoreQueryFilters().AnyAsync(ct))
        {
            // Direction +1 puts stock in, -1 takes stock out. The service layer
            // multiplies Quantity by Direction to roll up the on-hand figure.
            (string Name, string Code, int Direction)[] types =
            [
                ("Opening Balance",  "OPENING", +1),
                ("Purchase",         "PURCHASE", +1),
                ("Return",           "RETURN",   +1),
                ("Adjustment In",    "ADJ_IN",   +1),
                ("Issue",            "ISSUE",    -1),
                ("Write-off",        "WRITEOFF", -1),
                ("Adjustment Out",   "ADJ_OUT",  -1),
            ];
            for (var i = 0; i < types.Length; i++)
            {
                db.StockMovementTypes.Add(new StockMovementType
                {
                    Name = types[i].Name,
                    Code = types[i].Code,
                    Direction = types[i].Direction,
                    DisplayOrder = i + 1,
                });
            }
        }

        await db.SaveChangesAsync(ct);
    }

    private static async Task SeedFinanceLookupsAsync(ApplicationDbContext db,
CancellationToken ct)
```

What gets seeded:

- ItemCategories — Books (BOOK), Uniforms (UNIF), Stationery (STAT), Sports Equipment (SPRT), Cleaning Supplies (CLN), Food (FOOD), Furniture (FURN), ICT Equipment (ICT), Medical Supplies (MED), Other (OTH).
- UnitsOfMeasure — Each (EA), Piece (PC), Pack (PK), Box (BOX), Carton (CTN), Set (SET), Bag (BAG), Kilogram (KG), Litre (L), Metre (M).
- StockMovementTypes — Opening Balance (OPENING, +1), Purchase (PURCHASE, +1), Return (RETURN, +1), Adjustment In (ADJ_IN, +1), Issue (ISSUE, -1), Write-off (WRITEOFF, -1), Adjustment Out (ADJ_OUT, -1).

9. The Razor pages

```
src/NaijaPrimeSchool.Web/Components/Pages/Inventory/
├── StoreDashboard.razor          <- /store
├── StoreItems.razor              <- /store/items
├── CreateStoreItem.razor         <- /store/items/new
├── StoreItemDetail.razor         <- /store/items/{id}
├── StockMovements.razor          <- /store/movements
├── RecordStockMovement.razor     <- /store/movements/new
└── Suppliers.razor               <- /store/suppliers
```

Every page is gated to SuperAdmin + HeadTeacher + SchoolStoreKeeper. Sprint 7 is the first sprint where the SchoolStoreKeeper role moves from placeholder navigation to a fully realised workspace.

9.1 StoreDashboard.razor

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Inventory/StoreDashboard.razor

```
@page "/store"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher},
{Roles.SchoolStoreKeeper}")]
@rendermode InteractiveServer

@inject IStockMovementService MovementService
@inject NavigationManager Nav

<PageTitle>Store dashboard · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>Store dashboard</h1>
        <p>Items on hand, low-stock alerts, and a snapshot of stock movements this
month.</p>
    </div>
    <div class="nps-page-actions">
        <RadzenButton Text="Record movement" Icon="add" ButtonStyle="ButtonStyle.Primary"
                      Click="@(() => Nav.NavigateTo("/store/movements/new"))" />
    </div>
</div>

@if (loading || summary is null)
{
    <p>Loading dashboard...</p>
}
else
{
    <RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
        <div class="nps-stat-grid">
            <div class="nps-stat-card">
                <div class="nps-stat-icon"><RadzenIcon Icon="inventory_2" /></div>
                <div>
                    <div class="nps-stat-label">Items in catalog</div>
                    <div class="nps-stat-value">@summary.ActiveItems /
@summary.TotalItems</div>
                </div>
```

```
            </div>
            <div class="nps-stat-card">
                <div class="nps-stat-icon" style="background: #fde2e2; color: #d64545;">
                    <RadzenIcon Icon="warning" />
                </div>
                <div>
                    <div class="nps-stat-label">Below reorder level</div>
                    <div class="nps-stat-value">@summary.ItemsBelowReorder</div>
                </div>
            </div>
            <div class="nps-stat-card">
                <div class="nps-stat-icon"><RadzenIcon Icon="account_balance_wallet"
/></div>
                <div>
                    <div class="nps-stat-label">Stock value</div>
                    <div class="nps-stat-value">@summary.StockValue.ToString("N2")</div>
                </div>
            </div>
            <div class="nps-stat-card">
                <div class="nps-stat-icon"><RadzenIcon Icon="trending_up" /></div>
                <div>
                    <div class="nps-stat-label">This month: in / out</div>
                    <div class="nps-stat-value">@summary.InboundThisMonth /
@summary.OutboundThisMonth</div>
                </div>
            </div>
        </div>
    </RadzenCard>

    <div class="nps-form-grid">
        <RadzenCard class="nps-card">
            <h3 style="margin: 0 0 1rem;">Low stock — needs replenishing</h3>
            @if (summary.LowStockItems.Count == 0)
            {
                <p style="color: var(--nps-ink-500);">Every item is currently above its
reorder level.</p>
            }
            else
            {
                <RadzenDataGrid TItem="StoreItemDto" Data="@summary.LowStockItems"
AllowPaging="false">
                    <Columns>
                        <RadzenDataGridColumn TItem="StoreItemDto" Title="Item">
                            <Template Context="i">
                                <strong>@i.Name</strong>
                                @if (!string.IsNullOrEmpty(i.Sku))
                                {
                                    <br />
                                    <small style="color: var(--nps-ink-500);">SKU
@i.Sku</small>
                                }
                            </Template>
                        </RadzenDataGridColumn>
                        <RadzenDataGridColumn TItem="StoreItemDto" Title="On hand"
Width="120px">
```

```
                            <Template Context="i">
                                <strong>@i.QuantityOnHand.ToString("N2")</strong>
@i.UnitOfMeasureCode
                            </Template>
                        </RadzenDataGridColumn>
                        <RadzenDataGridColumn TItem="StoreItemDto" Title="Reorder at"
Width="120px">
                            <Template Context="i">@i.ReorderLevel.ToString("N2")</Template>
                        </RadzenDataGridColumn>
                        <RadzenDataGridColumn TItem="StoreItemDto" Title="Open"
Width="90px" Sortable="false"
                                              TextAlign="TextAlign.Right">
                            <Template Context="i">
                                <RadzenButton Icon="open_in_new" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                              ButtonStyle="ButtonStyle.Light"
                                              Click="@(() =>
Nav.NavigateTo($"/store/items/{i.Id}"))" />
                            </Template>
                        </RadzenDataGridColumn>
                    </Columns>
                </RadzenDataGrid>
            }
        </RadzenCard>
        <RadzenCard class="nps-card">
            <h3 style="margin: 0 0 1rem;">Stock value by category</h3>
            @if (summary.ByCategory.Count == 0)
            {
                <p style="color: var(--nps-ink-500);">No items yet.</p>
            }
            else
            {
                <RadzenDataGrid TItem="CategoryStockDto" Data="@summary.ByCategory"
AllowPaging="false">
                    <Columns>
                        <RadzenDataGridColumn TItem="CategoryStockDto" Title="Category"
Property="ItemCategoryName" />
                        <RadzenDataGridColumn TItem="CategoryStockDto" Title="Items"
Width="100px" Property="ItemCount" />
                        <RadzenDataGridColumn TItem="CategoryStockDto" Title="Value"
Width="160px">
                            <Template
Context="c"><strong>@c.StockValue.ToString("N2")</strong></Template>
                        </RadzenDataGridColumn>
                    </Columns>
                </RadzenDataGrid>
            }
        </RadzenCard>
    </div>

    <RadzenCard class="nps-card" Style="margin-top: 1rem;">
        <h3 style="margin: 0 0 1rem;">Recent movements</h3>
        @if (summary.RecentMovements.Count == 0)
        {
            <p style="color: var(--nps-ink-500);">No movements recorded yet.</p>
```

```
        }
        else
        {
            <RadzenDataGrid TItem="StockMovementDto" Data="@summary.RecentMovements"
AllowPaging="false">
                <Columns>
                    <RadzenDataGridColumn TItem="StockMovementDto" Title="Movement">
                        <Template Context="m">
                            <strong>@m.MovementNumber</strong>
                            <br />
                            <small style="color: var(--nps-
ink-500);">@m.MovedOn.ToString("d MMM yyyy")</small>
                        </Template>
                    </RadzenDataGridColumn>
                    <RadzenDataGridColumn TItem="StockMovementDto" Title="Item">
                        <Template Context="m">
                            <strong>@m.StoreItemName</strong>
                            @if (!string.IsNullOrEmpty(m.StoreItemSku))
                            {
                                <br />
                                <small style="color: var(--nps-ink-500);">SKU
@m.StoreItemSku</small>
                            }
                        </Template>
                    </RadzenDataGridColumn>
                    <RadzenDataGridColumn TItem="StockMovementDto" Title="Type"
Width="140px">
                        <Template Context="m">
                            <RadzenBadge Text="@m.StockMovementTypeName"
                                         BadgeStyle="@(m.Direction == +1 ?
BadgeStyle.Success : BadgeStyle.Warning)"
                                         Variant="Variant.Flat" />
                        </Template>
                    </RadzenDataGridColumn>
                    <RadzenDataGridColumn TItem="StockMovementDto" Title="Quantity"
Width="140px">
                        <Template Context="m">
                            <strong>
                                @(m.Direction == +1 ? "+" : "−")@m.Quantity.ToString("N2")
                            </strong> @m.UnitOfMeasureCode
                        </Template>
                    </RadzenDataGridColumn>
                </Columns>
            </RadzenDataGrid>
        }
    </RadzenCard>
}

@code {
    private StoreSummaryDto? summary;
    private bool loading;

    protected override async Task OnInitializedAsync()
    {
        loading = true;
```

```
        try { summary = await MovementService.GetStoreSummaryAsync(); }
        finally { loading = false; }
    }
}
```

9.2 StoreItems.razor

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Inventory/StoreItems.razor

```
@page "/store/items"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher},
{Roles.SchoolStoreKeeper}")]
@rendermode InteractiveServer

@inject IStoreItemService ItemService
@inject ILookupService LookupService
@inject NavigationManager Nav
@inject DialogService Dialog
@inject NotificationService Notification

<PageTitle>Store catalog · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>Store catalog</h1>
        <p>Every item the storekeeper tracks — books, uniforms, stationery, cleaning
supplies, food, and the rest.</p>
    </div>
    <div class="nps-page-actions">
        <RadzenButton Text="New item" Icon="add" ButtonStyle="ButtonStyle.Primary"
                      Click="@(() => Nav.NavigateTo("/store/items/new"))" />
    </div>
</div>

<RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
    <div class="nps-filter-bar">
        <div class="nps-field">
            <label>Search</label>
            <RadzenTextBox @bind-Value="search" Placeholder="Name or SKU"
                           Change="@(_ => LoadAsync())" Style="min-width: 240px;" />
        </div>
        <div class="nps-field">
            <label>Category</label>
            <RadzenDropDown TValue="Guid?" Data="@categories" TextProperty="Name"
ValueProperty="Id"
                            @bind-Value="categoryId" AllowClear="true" Placeholder="All
categories"
                            Change="@(_ => LoadAsync())" Style="min-width: 200px;" />
        </div>
        <div class="nps-field">
            <label>Status</label>
            <RadzenDropDown TValue="bool?" Data="@statusOptions" TextProperty="Text"
ValueProperty="Value"
                            @bind-Value="isActive" AllowClear="true" Placeholder="All"
```

```
                            Change="@(_ => LoadAsync())" Style="min-width: 160px;" />
        </div>
        <div class="nps-field">
            <label>Low stock only</label>
            <RadzenSwitch @bind-Value="lowStockOnly" Change="@(_ => LoadAsync())" />
        </div>
    </div>
</RadzenCard>

<RadzenCard class="nps-card">
    <RadzenDataGrid TItem="StoreItemDto" Data="@items" IsLoading="@loading"
                    AllowPaging="true" PageSize="25" EmptyText="No items match your
filters.">
        <Columns>
            <RadzenDataGridColumn TItem="StoreItemDto" Title="Item">
                <Template Context="i">
                    <strong>@i.Name</strong>
                    @if (!string.IsNullOrEmpty(i.Sku))
                    {
                        <br />
                        <small style="color: var(--nps-ink-500);">SKU @i.Sku</small>
                    }
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="StoreItemDto" Title="Category"
Property="ItemCategoryName" Width="160px" />
            <RadzenDataGridColumn TItem="StoreItemDto" Title="On hand" Width="140px">
                <Template Context="i">
                    <strong>@i.QuantityOnHand.ToString("N2")</strong> @i.UnitOfMeasureCode
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="StoreItemDto" Title="Reorder" Width="120px">
                <Template Context="i">@i.ReorderLevel.ToString("N2")</Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="StoreItemDto" Title="Last cost" Width="140px">
                <Template Context="i">@i.LastUnitCost.ToString("N2")</Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="StoreItemDto" Title="Status" Width="120px">
                <Template Context="i">
                    @if (!i.IsActive)
                    {
                        <RadzenBadge Text="Inactive" BadgeStyle="BadgeStyle.Secondary"
Variant="Variant.Flat" />
                    }
                    else if (i.IsBelowReorder)
                    {
                        <RadzenBadge Text="Low" BadgeStyle="BadgeStyle.Warning"
Variant="Variant.Flat" />
                    }
                    else
                    {
                        <RadzenBadge Text="In stock" BadgeStyle="BadgeStyle.Success"
Variant="Variant.Flat" />
                    }
                </Template>
```

```
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="StoreItemDto" Title="Actions" Width="180px"
Sortable="false"
                                  TextAlign="TextAlign.Right">
                <Template Context="i">
                    <div class="nps-inline-actions">
                        <RadzenButton Icon="visibility" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                      ButtonStyle="ButtonStyle.Primary" title="Open"
                                      Click="@(() =>
Nav.NavigateTo($"/store/items/{i.Id}"))" />
                        <RadzenButton Icon="add" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                      ButtonStyle="ButtonStyle.Success" title="Record
movement"
                                      Click="@(() => Nav.NavigateTo($"/store/movements/new?
itemId={i.Id}"))" />
                        <RadzenButton Icon="delete" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                      ButtonStyle="ButtonStyle.Danger" title="Delete"
                                      Click="@(() => DeleteAsync(i))" />
                    </div>
                </Template>
            </RadzenDataGridColumn>
        </Columns>
    </RadzenDataGrid>
</RadzenCard>

@code {
    private IReadOnlyList<StoreItemDto> items = [];
    private IReadOnlyList<LookupDto> categories = [];

    private bool loading;
    private string? search;
    private Guid? categoryId;
    private bool? isActive;
    private bool lowStockOnly;

    private record StatusOption(string Text, bool? Value);
    private readonly List<StatusOption> statusOptions =
    [
        new StatusOption("Active", true),
        new StatusOption("Inactive", false),
    ];

    protected override async Task OnInitializedAsync()
    {
        categories = await LookupService.GetItemCategoriesAsync();
        isActive = true;
        await LoadAsync();
    }

    private async Task LoadAsync()
    {
        loading = true;
```

```
        try
        {
            items = await ItemService.ListAsync(new StoreItemFilter
            {
                Search = search,
                ItemCategoryId = categoryId,
                IsActive = isActive,
                OnlyBelowReorder = lowStockOnly ? true : null,
            });
        }
        finally
        {
            loading = false;
            StateHasChanged();
        }
    }

    private async Task DeleteAsync(StoreItemDto item)
    {
        var confirm = await Dialog.Confirm(
            $"Delete '{item.Name}'? Only items with no movement history can be deleted.",
            "Delete item",
            new ConfirmOptions { OkButtonText = "Delete", CancelButtonText = "Cancel" });
        if (confirm != true) return;

        var result = await ItemService.SoftDeleteAsync(item.Id);
        if (result.Succeeded)
        {
            Notification.Notify(NotificationSeverity.Success, "Deleted", $"'{item.Name}'
removed.");
            await LoadAsync();
        }
        else
        {
            Notification.Notify(NotificationSeverity.Error, "Failed",
                string.Join("; ", result.Errors));
        }
    }
}
```

9.3 CreateStoreItem.razor

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Inventory/CreateStoreItem.razor

```
@page "/store/items/new"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher},
{Roles.SchoolStoreKeeper}")]
@rendermode InteractiveServer

@inject IStoreItemService ItemService
@inject ILookupService LookupService
@inject NavigationManager Nav
@inject NotificationService Notification
```

```
<PageTitle>New store item · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>New store item</h1>
        <p>Add an item to the catalog. Set the opening quantity to seed the stock on
hand.</p>
    </div>
    <div class="nps-page-actions">
        <RadzenButton Text="Back" Icon="arrow_back" Variant="Variant.Flat"
                      ButtonStyle="ButtonStyle.Light" Click="@(() =>
Nav.NavigateTo("/store/items"))" />
    </div>
</div>

<RadzenCard class="nps-card">
    <RadzenTemplateForm TItem="ItemFormModel" Data="@form" Submit="@SaveAsync">
        <DataAnnotationsValidator />

        <h3 style="margin: 0 0 1rem;">Identity</h3>
        <div class="nps-form-grid">
            <div class="nps-field nps-span-2">
                <label>Name *</label>
                <RadzenTextBox @bind-Value="form.Name" Placeholder="e.g. English Textbook —
Primary 4" />
                <ValidationMessage For="() => form.Name" />
            </div>
            <div class="nps-field">
                <label>SKU</label>
                <RadzenTextBox @bind-Value="form.Sku" Placeholder="Optional unique code" />
            </div>
            <div class="nps-field">
                <label>Category *</label>
                <RadzenDropDown TValue="Guid?" Data="@categories" TextProperty="Name"
ValueProperty="Id"
                                @bind-Value="form.ItemCategoryId" Placeholder="Pick a
category" />
            </div>
            <div class="nps-field">
                <label>Unit of measure *</label>
                <RadzenDropDown TValue="Guid?" Data="@units" TextProperty="Name"
ValueProperty="Id"
                                @bind-Value="form.UnitOfMeasureId" Placeholder="Pick a
unit" />
            </div>
            <div class="nps-field">
                <label>Reorder level</label>
                <RadzenNumeric TValue="decimal" @bind-Value="form.ReorderLevel" Min="0"
Format="N2" />
                <small style="color: var(--nps-ink-500);">
                    Low-stock badge appears when on-hand falls to or below this number.
                </small>
            </div>
            <div class="nps-field nps-span-2">
                <label>Description</label>
```

```
                <RadzenTextArea @bind-Value="form.Description" Rows="2" />
            </div>
        </div>

        <h3 style="margin: 1.5rem 0 1rem;">Opening balance (optional)</h3>
        <div class="nps-form-grid">
            <div class="nps-field">
                <label>Opening quantity</label>
                <RadzenNumeric TValue="decimal" @bind-Value="form.OpeningQuantity" Min="0"
Format="N2" />
                <small style="color: var(--nps-ink-500);">
                    Recorded as a single "Opening Balance" movement.
                </small>
            </div>
            <div class="nps-field">
                <label>Opening unit cost</label>
                <RadzenNumeric TValue="decimal" @bind-Value="form.OpeningUnitCost" Min="0"
Format="N2" />
                <small style="color: var(--nps-ink-500);">
                    Feeds the stock-value figure on the dashboard.
                </small>
            </div>
        </div>

        <div class="nps-form-actions">
            <RadzenButton Text="Cancel" ButtonType="ButtonType.Button"
                          ButtonStyle="ButtonStyle.Light" Variant="Variant.Flat"
                          Click="@(() => Nav.NavigateTo("/store/items"))" />
            <RadzenButton Text="@(saving ? "Saving..." : "Save item")"
                          Icon="save" ButtonType="ButtonType.Submit"
                          ButtonStyle="ButtonStyle.Primary" Disabled="@saving" />
        </div>
    </RadzenTemplateForm>
</RadzenCard>

@code {
    private ItemFormModel form = new();
    private bool saving;

    private IReadOnlyList<LookupDto> categories = [];
    private IReadOnlyList<LookupDto> units = [];

    private class ItemFormModel
    {
        [System.ComponentModel.DataAnnotations.Required,
System.ComponentModel.DataAnnotations.StringLength(120)]
        public string Name { get; set; } = string.Empty;

        [System.ComponentModel.DataAnnotations.StringLength(60)]
        public string? Sku { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(500)]
        public string? Description { get; set; }

        public Guid? ItemCategoryId { get; set; }
```

```
        public Guid? UnitOfMeasureId { get; set; }
        public decimal ReorderLevel { get; set; }
        public decimal OpeningQuantity { get; set; }
        public decimal OpeningUnitCost { get; set; }
    }

    protected override async Task OnInitializedAsync()
    {
        categories = await LookupService.GetItemCategoriesAsync();
        units = await LookupService.GetUnitsOfMeasureAsync();
    }

    private async Task SaveAsync()
    {
        if (form.ItemCategoryId is null || form.UnitOfMeasureId is null)
        {
            Notification.Notify(NotificationSeverity.Warning, "Missing fields",
                "Pick a category and a unit of measure.");
            return;
        }

        saving = true;
        try
        {
            var result = await ItemService.CreateAsync(new CreateStoreItemRequest
            {
                Name = form.Name,
                Sku = form.Sku,
                Description = form.Description,
                ItemCategoryId = form.ItemCategoryId.Value,
                UnitOfMeasureId = form.UnitOfMeasureId.Value,
                ReorderLevel = form.ReorderLevel,
                OpeningQuantity = form.OpeningQuantity,
                OpeningUnitCost = form.OpeningUnitCost,
                IsActive = true,
            });

            if (result.Succeeded && result.Data != Guid.Empty)
            {
                Notification.Notify(NotificationSeverity.Success, "Saved", $"'{form.Name}'
added.");
                Nav.NavigateTo($"/store/items/{result.Data}");
            }
            else
            {
                Notification.Notify(NotificationSeverity.Error, "Failed",
                    string.Join("; ", result.Errors));
            }
        }
        finally
        {
            saving = false;
        }
    }
}
```

9.4 StoreItemDetail.razor

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Inventory/StoreItemDetail.razor

```
@page "/store/items/{Id:guid}"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher},
{Roles.SchoolStoreKeeper}")]
@rendermode InteractiveServer

@inject IStoreItemService ItemService
@inject ILookupService LookupService
@inject NavigationManager Nav
@inject DialogService Dialog
@inject NotificationService Notification

<PageTitle>Item · Naija Prime School</PageTitle>

@if (loading)
{
    <p>Loading item...</p>
}
else if (detail is null)
{
    <RadzenAlert Severity="AlertSeverity.Warning">Item not found.</RadzenAlert>
}
else
{
    <div class="nps-page-header">
        <div>
            <h1>@detail.Item.Name</h1>
            <p>
                @detail.Item.ItemCategoryName ·
                on hand <strong>@detail.Item.QuantityOnHand.ToString("N2")</strong>
@detail.Item.UnitOfMeasureCode
                @if (!string.IsNullOrEmpty(detail.Item.Sku))
                {
                    <span> · SKU @detail.Item.Sku</span>
                }
            </p>
        </div>
        <div class="nps-page-actions">
            @if (detail.Item.IsBelowReorder && detail.Item.IsActive)
            {
                <RadzenBadge Text="Low stock" BadgeStyle="BadgeStyle.Warning"
Variant="Variant.Flat" />
            }
            <RadzenButton Text="Record movement" Icon="add"
ButtonStyle="ButtonStyle.Primary"
                          Click="@(() => Nav.NavigateTo($"/store/movements/new?
itemId={Id}"))" />
            <RadzenButton Text="Back" Icon="arrow_back" Variant="Variant.Flat"
                          ButtonStyle="ButtonStyle.Light" Click="@(() =>
Nav.NavigateTo("/store/items"))" />
```

```
        </div>
    </div>

    <RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
        <div class="nps-stat-grid">
            <div class="nps-stat-card">
                <div class="nps-stat-icon"><RadzenIcon Icon="inventory" /></div>
                <div>
                    <div class="nps-stat-label">On hand</div>
                    <div class="nps-stat-value">@detail.Item.QuantityOnHand.ToString("N2")
@detail.Item.UnitOfMeasureCode</div>
                </div>
            </div>
            <div class="nps-stat-card">
                <div class="nps-stat-icon"><RadzenIcon Icon="warning_amber" /></div>
                <div>
                    <div class="nps-stat-label">Reorder level</div>
                    <div
class="nps-stat-value">@detail.Item.ReorderLevel.ToString("N2")</div>
                </div>
            </div>
            <div class="nps-stat-card">
                <div class="nps-stat-icon"><RadzenIcon Icon="payments" /></div>
                <div>
                    <div class="nps-stat-label">Last unit cost</div>
                    <div
class="nps-stat-value">@detail.Item.LastUnitCost.ToString("N2")</div>
                </div>
            </div>
            <div class="nps-stat-card">
                <div class="nps-stat-icon"><RadzenIcon Icon="account_balance_wallet"
/></div>
                <div>
                    <div class="nps-stat-label">Stock value</div>
                    <div
class="nps-stat-value">@detail.Item.StockValue.ToString("N2")</div>
                </div>
            </div>
        </div>
    </RadzenCard>

    <RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
        <h3 style="margin: 0 0 1rem;">Edit catalog fields</h3>
        <RadzenTemplateForm TItem="ItemEditModel" Data="@form" Submit="@SaveAsync">
            <DataAnnotationsValidator />
            <div class="nps-form-grid">
                <div class="nps-field nps-span-2">
                    <label>Name *</label>
                    <RadzenTextBox @bind-Value="form.Name" />
                </div>
                <div class="nps-field">
                    <label>SKU</label>
                    <RadzenTextBox @bind-Value="form.Sku" />
                </div>
                <div class="nps-field">
```

```
                    <label>Category *</label>
                    <RadzenDropDown TValue="Guid?" Data="@categories" TextProperty="Name"
ValueProperty="Id"
                                    @bind-Value="form.ItemCategoryId" />
                </div>
                <div class="nps-field">
                    <label>Unit of measure *</label>
                    <RadzenDropDown TValue="Guid?" Data="@units" TextProperty="Name"
ValueProperty="Id"
                                    @bind-Value="form.UnitOfMeasureId" />
                </div>
                <div class="nps-field">
                    <label>Reorder level</label>
                    <RadzenNumeric TValue="decimal" @bind-Value="form.ReorderLevel" Min="0"
Format="N2" />
                </div>
                <div class="nps-field nps-span-2">
                    <label>Description</label>
                    <RadzenTextArea @bind-Value="form.Description" Rows="2" />
                </div>
            </div>
            <div class="nps-form-actions">
                @if (detail.Item.IsActive)
                {
                    <RadzenButton Text="Deactivate" Icon="block" Variant="Variant.Flat"
                                  ButtonStyle="ButtonStyle.Warning"
ButtonType="ButtonType.Button"
                                  Click="@(() => SetActiveAsync(false))" />
                }
                else
                {
                    <RadzenButton Text="Activate" Icon="check_circle"
Variant="Variant.Flat"
                                  ButtonStyle="ButtonStyle.Success"
ButtonType="ButtonType.Button"
                                  Click="@(() => SetActiveAsync(true))" />
                }
                <RadzenButton Text="@(saving ? "Saving..." : "Save changes")"
                              Icon="save" ButtonType="ButtonType.Submit"
                              ButtonStyle="ButtonStyle.Primary" Disabled="@saving" />
            </div>
        </RadzenTemplateForm>
    </RadzenCard>

    <RadzenCard class="nps-card">
        <h3 style="margin: 0 0 1rem;">Movement history (@detail.Movements.Count
latest)</h3>
        @if (detail.Movements.Count == 0)
        {
            <p style="color: var(--nps-ink-500);">No movements recorded for this item
yet.</p>
        }
        else
        {
            <RadzenDataGrid TItem="StockMovementDto" Data="@detail.Movements"
```

```
AllowPaging="true" PageSize="25">
                <Columns>
                    <RadzenDataGridColumn TItem="StockMovementDto" Title="Number">
                        <Template Context="m">
                            <strong>@m.MovementNumber</strong>
                            <br />
                            <small style="color: var(--nps-
ink-500);">@m.MovedOn.ToString("d MMM yyyy")</small>
                        </Template>
                    </RadzenDataGridColumn>
                    <RadzenDataGridColumn TItem="StockMovementDto" Title="Type"
Width="160px">
                        <Template Context="m">
                            <RadzenBadge Text="@m.StockMovementTypeName"
                                         BadgeStyle="@(m.Direction == +1 ?
BadgeStyle.Success : BadgeStyle.Warning)"
                                         Variant="Variant.Flat" />
                        </Template>
                    </RadzenDataGridColumn>
                    <RadzenDataGridColumn TItem="StockMovementDto" Title="Quantity"
Width="160px">
                        <Template Context="m">
                            <strong>
                                @(m.Direction == +1 ? "+" : "−")@m.Quantity.ToString("N2")
                            </strong> @m.UnitOfMeasureCode
                        </Template>
                    </RadzenDataGridColumn>
                    <RadzenDataGridColumn TItem="StockMovementDto" Title="Counter-party">
                        <Template Context="m">
                            @if (m.ReceivedFromSupplierName is { Length: > 0 })
                            {
                                <text>From
<strong>@m.ReceivedFromSupplierName</strong></text>
                            }
                            else if (m.IssuedToStudentName is { Length: > 0 })
                            {
                                <text>To pupil
<strong>@m.IssuedToStudentName</strong></text>
                            }
                            else if (m.IssuedToSchoolClassName is { Length: > 0 })
                            {
                                <text>To class
<strong>@m.IssuedToSchoolClassName</strong></text>
                            }
                            else if (m.IssuedToUserName is { Length: > 0 })
                            {
                                <text>To staff <strong>@m.IssuedToUserName</strong></text>
                            }
                            else
                            {
                                <span style="color: var(--nps-ink-500);">—</span>
                            }
                        </Template>
                    </RadzenDataGridColumn>
                    <RadzenDataGridColumn TItem="StockMovementDto" Title="Reference"
```

```
Property="Reference" />
                    <RadzenDataGridColumn TItem="StockMovementDto" Title="Total cost"
Width="140px">
                        <Template Context="m">@m.TotalCost.ToString("N2")</Template>
                    </RadzenDataGridColumn>
                </Columns>
            </RadzenDataGrid>
        }
    </RadzenCard>
}

@code {
    [Parameter] public Guid Id { get; set; }

    private StoreItemDetailDto? detail;
    private bool loading = true;
    private bool saving;

    private IReadOnlyList<LookupDto> categories = [];
    private IReadOnlyList<LookupDto> units = [];

    private ItemEditModel form = new();

    private class ItemEditModel
    {
        [System.ComponentModel.DataAnnotations.Required,
System.ComponentModel.DataAnnotations.StringLength(120)]
        public string Name { get; set; } = string.Empty;

        [System.ComponentModel.DataAnnotations.StringLength(60)]
        public string? Sku { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(500)]
        public string? Description { get; set; }

        public Guid? ItemCategoryId { get; set; }
        public Guid? UnitOfMeasureId { get; set; }
        public decimal ReorderLevel { get; set; }
    }

    protected override async Task OnInitializedAsync()
    {
        categories = await LookupService.GetItemCategoriesAsync();
        units = await LookupService.GetUnitsOfMeasureAsync();
        await LoadAsync();
    }

    private async Task LoadAsync()
    {
        loading = true;
        try
        {
            detail = await ItemService.GetByIdAsync(Id);
            if (detail is not null)
            {
```

```
                form = new ItemEditModel
                {
                    Name = detail.Item.Name,
                    Sku = detail.Item.Sku,
                    Description = detail.Item.Description,
                    ItemCategoryId = detail.Item.ItemCategoryId,
                    UnitOfMeasureId = detail.Item.UnitOfMeasureId,
                    ReorderLevel = detail.Item.ReorderLevel,
                };
            }
        }
        finally
        {
            loading = false;
        }
    }

    private async Task SaveAsync()
    {
        if (form.ItemCategoryId is null || form.UnitOfMeasureId is null) return;

        saving = true;
        try
        {
            var result = await ItemService.UpdateAsync(new UpdateStoreItemRequest
            {
                Id = Id,
                Name = form.Name,
                Sku = form.Sku,
                Description = form.Description,
                ItemCategoryId = form.ItemCategoryId.Value,
                UnitOfMeasureId = form.UnitOfMeasureId.Value,
                ReorderLevel = form.ReorderLevel,
            });
            if (result.Succeeded)
            {
                Notification.Notify(NotificationSeverity.Success, "Saved", "Item
updated.");
                await LoadAsync();
            }
            else
            {
                Notification.Notify(NotificationSeverity.Error, "Failed",
                    string.Join("; ", result.Errors));
            }
        }
        finally
        {
            saving = false;
        }
    }

    private async Task SetActiveAsync(bool active)
    {
        var result = await ItemService.SetActiveAsync(Id, active, CancellationToken.None);
```

```
        if (result.Succeeded)
        {
            Notification.Notify(NotificationSeverity.Success, "Updated",
                active ? "Item activated." : "Item deactivated.");
            await LoadAsync();
        }
        else
        {
            Notification.Notify(NotificationSeverity.Error, "Failed",
                string.Join("; ", result.Errors));
        }
    }
}
```

9.5 StockMovements.razor

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Inventory/StockMovements.razor

```
@page "/store/movements"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher},
{Roles.SchoolStoreKeeper}")]
@rendermode InteractiveServer

@inject IStockMovementService MovementService
@inject ILookupService LookupService
@inject NavigationManager Nav

<PageTitle>Stock movements · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>Stock movements</h1>
        <p>Every receipt, issuance, return, write-off, and adjustment that touched the
store.</p>
    </div>
    <div class="nps-page-actions">
        <RadzenButton Text="Record movement" Icon="add" ButtonStyle="ButtonStyle.Primary"
                      Click="@(() => Nav.NavigateTo("/store/movements/new"))" />
    </div>
</div>

<RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
    <div class="nps-filter-bar">
        <div class="nps-field">
            <label>Search</label>
            <RadzenTextBox @bind-Value="search" Placeholder="Movement no, item, reference"
                           Change="@(_ => LoadAsync())" Style="min-width: 280px;" />
        </div>
        <div class="nps-field">
            <label>Type</label>
            <RadzenDropDown TValue="Guid?" Data="@types" TextProperty="Name"
ValueProperty="Id"
                            @bind-Value="typeId" AllowClear="true" Placeholder="All types"
                            Change="@(_ => LoadAsync())" Style="min-width: 180px;" />
```

```
        </div>
        <div class="nps-field">
            <label>Direction</label>
            <RadzenDropDown TValue="int?" Data="@directionOptions" TextProperty="Text"
ValueProperty="Value"
                            @bind-Value="direction" AllowClear="true" Placeholder="Any"
                            Change="@(_ => LoadAsync())" Style="min-width: 140px;" />
        </div>
        <div class="nps-field">
            <label>From</label>
            <RadzenDatePicker TValue="DateTime?" @bind-Value="fromDate" DateFormat="d MMM
yyyy"
                              Change="@(_ => LoadAsync())" Style="min-width: 160px;" />
        </div>
        <div class="nps-field">
            <label>To</label>
            <RadzenDatePicker TValue="DateTime?" @bind-Value="toDate" DateFormat="d MMM
yyyy"
                              Change="@(_ => LoadAsync())" Style="min-width: 160px;" />
        </div>
    </div>
</RadzenCard>

<RadzenCard class="nps-card">
    <RadzenDataGrid TItem="StockMovementDto" Data="@movements" IsLoading="@loading"
                    AllowPaging="true" PageSize="25" EmptyText="No movements match your
filters.">
        <Columns>
            <RadzenDataGridColumn TItem="StockMovementDto" Title="Movement">
                <Template Context="m">
                    <strong>@m.MovementNumber</strong>
                    <br />
                    <small style="color: var(--nps-ink-500);">@m.MovedOn.ToString("d MMM
yyyy")</small>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="StockMovementDto" Title="Item">
                <Template Context="m">
                    <strong>@m.StoreItemName</strong>
                    @if (!string.IsNullOrEmpty(m.StoreItemSku))
                    {
                        <br />
                        <small style="color: var(--nps-ink-500);">SKU
@m.StoreItemSku</small>
                    }
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="StockMovementDto" Title="Type" Width="160px">
                <Template Context="m">
                    <RadzenBadge Text="@m.StockMovementTypeName"
                                 BadgeStyle="@(m.Direction == +1 ? BadgeStyle.Success :
BadgeStyle.Warning)"
                                 Variant="Variant.Flat" />
                </Template>
            </RadzenDataGridColumn>
```

```
            <RadzenDataGridColumn TItem="StockMovementDto" Title="Quantity" Width="160px">
                <Template Context="m">
                    <strong>
                        @(m.Direction == +1 ? "+" : "−")@m.Quantity.ToString("N2")
                    </strong> @m.UnitOfMeasureCode
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="StockMovementDto" Title="Counter-party">
                <Template Context="m">
                    @if (m.ReceivedFromSupplierName is { Length: > 0 })
                    {
                        <text>From <strong>@m.ReceivedFromSupplierName</strong></text>
                    }
                    else if (m.IssuedToStudentName is { Length: > 0 })
                    {
                        <div class="nps-cell-with-avatar">
                            <StudentAvatar PhotoUrl="@m.IssuedToStudentPhotoUrl"
                                           FirstName="@m.IssuedToStudentFirstName"
                                           LastName="@m.IssuedToStudentLastName" />
                            <div style="display:flex; flex-direction:column;">
                                <strong>@m.IssuedToStudentName</strong>
                                <small style="color: var(--nps-
ink-500);">@m.IssuedToStudentAdmissionNumber</small>
                            </div>
                        </div>
                    }
                    else if (m.IssuedToSchoolClassName is { Length: > 0 })
                    {
                        <text>To class <strong>@m.IssuedToSchoolClassName</strong></text>
                    }
                    else if (m.IssuedToUserName is { Length: > 0 })
                    {
                        <text>To staff <strong>@m.IssuedToUserName</strong></text>
                    }
                    else
                    {
                        <span style="color: var(--nps-ink-500);">—</span>
                    }
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="StockMovementDto" Title="Total cost"
Width="140px">
                <Template Context="m">@m.TotalCost.ToString("N2")</Template>
            </RadzenDataGridColumn>
        </Columns>
    </RadzenDataGrid>
</RadzenCard>

@code {
    private IReadOnlyList<StockMovementDto> movements = [];
    private IReadOnlyList<LookupDto> types = [];

    private bool loading;
    private string? search;
    private Guid? typeId;
```

```
    private int? direction;
    private DateTime? fromDate;
    private DateTime? toDate;

    private record DirectionOption(string Text, int? Value);
    private readonly List<DirectionOption> directionOptions =
    [
        new DirectionOption("Inbound", +1),
        new DirectionOption("Outbound", -1),
    ];

    protected override async Task OnInitializedAsync()
    {
        types = await LookupService.GetStockMovementTypesAsync();
        await LoadAsync();
    }

    private async Task LoadAsync()
    {
        loading = true;
        try
        {
            movements = await MovementService.ListAsync(new StockMovementFilter
            {
                Search = search,
                StockMovementTypeId = typeId,
                Direction = direction,
                FromDate = fromDate.HasValue ? DateOnly.FromDateTime(fromDate.Value) :
null,
                ToDate = toDate.HasValue ? DateOnly.FromDateTime(toDate.Value) : null,
            });
        }
        finally
        {
            loading = false;
            StateHasChanged();
        }
    }
}
```

9.6 RecordStockMovement.razor

The most interesting page in the sprint. Picking a movement type flips the lower half of the form between an inbound panel (asks for a Supplier) and an outbound panel (asks for at most one of pupil / class / staff member). Both sides share the same upper half: item, type, date, quantity, unit cost, reference, notes.

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Inventory/RecordStockMovement.razor

```
@page "/store/movements/new"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher},
{Roles.SchoolStoreKeeper}")]
@rendermode InteractiveServer

@inject IStockMovementService MovementService
```

```
@inject ILookupService LookupService
@inject NavigationManager Nav
@inject NotificationService Notification

<PageTitle>Record stock movement · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>Record stock movement</h1>
        <p>One movement = one row in the audit log. Pick a type and the counter-party flips
into place.</p>
    </div>
    <div class="nps-page-actions">
        <RadzenButton Text="Back" Icon="arrow_back" Variant="Variant.Flat"
                      ButtonStyle="ButtonStyle.Light" Click="@(() =>
Nav.NavigateTo("/store/movements"))" />
    </div>
</div>

<RadzenCard class="nps-card">
    <h3 style="margin: 0 0 1rem;">Details</h3>
    <div class="nps-form-grid">
        <div class="nps-field nps-span-2">
            <label>Item *</label>
            <RadzenDropDown TValue="Guid?" Data="@items" TextProperty="Name"
ValueProperty="Id"
                            @bind-Value="itemId" Placeholder="Search and pick an item"
                            AllowFiltering="true"

FilterCaseSensitivity="FilterCaseSensitivity.CaseInsensitive" />
        </div>
        <div class="nps-field">
            <label>Movement type *</label>
            <RadzenDropDown TValue="Guid?" Data="@types" TextProperty="Name"
ValueProperty="Id"
                            @bind-Value="typeId" Placeholder="Pick a type"
                            Change="@(_ => UpdateDirection())" />
        </div>
        <div class="nps-field">
            <label>Date *</label>
            <RadzenDatePicker TValue="DateTime?" @bind-Value="movedOn"
                              DateFormat="d MMM yyyy" ShowTime="false" />
        </div>
        <div class="nps-field">
            <label>Quantity *</label>
            <RadzenNumeric TValue="decimal" @bind-Value="quantity" Min="0" Format="N3" />
        </div>
        <div class="nps-field">
            <label>Unit cost</label>
            <RadzenNumeric TValue="decimal" @bind-Value="unitCost" Min="0" Format="N2" />
        </div>
        <div class="nps-field">
            <label>Reference</label>
            <RadzenTextBox @bind-Value="reference" Placeholder="PO number, invoice,
requisition no" />
```

```
        </div>
        <div class="nps-field nps-span-2">
            <label>Notes</label>
            <RadzenTextArea @bind-Value="notes" Rows="2" />
        </div>
    </div>

    @if (selectedDirection == +1)
    {
        <h3 style="margin: 1.5rem 0 1rem;">Inbound — received from</h3>
        <div class="nps-form-grid">
            <div class="nps-field nps-span-2">
                <label>Supplier</label>
                <RadzenDropDown TValue="Guid?" Data="@suppliers" TextProperty="Name"
ValueProperty="Id"
                                @bind-Value="supplierId" AllowClear="true"
                                Placeholder="Pick a supplier (optional)"
                                AllowFiltering="true"

FilterCaseSensitivity="FilterCaseSensitivity.CaseInsensitive" />
            </div>
        </div>
    }
    else if (selectedDirection == -1)
    {
        <h3 style="margin: 1.5rem 0 1rem;">Outbound — issued to</h3>
        <p style="color: var(--nps-ink-500); margin: 0 0 1rem;">
            Pick at most one recipient. Leave all three empty for general consumption (e.g.
cleaning supplies used by the school).
        </p>
        <div class="nps-form-grid">
            <div class="nps-field">
                <label>Pupil</label>
                <RadzenDropDown TValue="Guid?" Data="@students" TextProperty="Name"
ValueProperty="Id"
                                @bind-Value="studentId" AllowClear="true" Placeholder="—"
                                AllowFiltering="true"

FilterCaseSensitivity="FilterCaseSensitivity.CaseInsensitive" />
            </div>
            <div class="nps-field">
                <label>Class</label>
                <RadzenDropDown TValue="Guid?" Data="@classes" TextProperty="Name"
ValueProperty="Id"
                                @bind-Value="schoolClassId" AllowClear="true"
Placeholder="—" />
            </div>
            <div class="nps-field nps-span-2">
                <label>Staff member</label>
                <RadzenDropDown TValue="Guid?" Data="@teachers" TextProperty="Name"
ValueProperty="Id"
                                @bind-Value="userId" AllowClear="true" Placeholder="—"
                                AllowFiltering="true"

FilterCaseSensitivity="FilterCaseSensitivity.CaseInsensitive" />
```

```
            </div>
        </div>
    }

    <div class="nps-form-actions">
        <RadzenButton Text="@(saving ? "Saving..." : "Save movement")"
                      Icon="save" ButtonStyle="ButtonStyle.Primary"
                      Disabled="@saving" Click="@SaveAsync" />
    </div>
</RadzenCard>

@code {
    [SupplyParameterFromQuery(Name = "itemId")] public Guid? QueryItemId { get; set; }

    private IReadOnlyList<LookupDto> items = [];
    private IReadOnlyList<LookupDto> types = [];
    private IReadOnlyList<LookupDto> suppliers = [];
    private IReadOnlyList<LookupDto> students = [];
    private IReadOnlyList<LookupDto> classes = [];
    private IReadOnlyList<LookupDto> teachers = [];

    private Guid? itemId;
    private Guid? typeId;
    private DateTime? movedOn = DateTime.Today;
    private decimal quantity;
    private decimal unitCost;
    private string? reference;
    private string? notes;

    private Guid? supplierId;
    private Guid? studentId;
    private Guid? schoolClassId;
    private Guid? userId;

    private bool saving;
    private int selectedDirection;
    private List<LookupDto> typesRaw = [];

    protected override async Task OnInitializedAsync()
    {
        items = await LookupService.GetStoreItemsAsync();
        var typeList = await LookupService.GetStockMovementTypesAsync();
        types = typeList;
        typesRaw = typeList.ToList();
        suppliers = await LookupService.GetActiveSuppliersAsync();
        students = await LookupService.GetStudentsAsync();
        classes = await LookupService.GetClassesForSessionAsync();
        teachers = await LookupService.GetTeachersAsync();

        if (QueryItemId.HasValue) itemId = QueryItemId;
    }

    private void UpdateDirection()
    {
        // Reset counter-party fields when the type changes.
```

```
        supplierId = null;
        studentId = null;
        schoolClassId = null;
        userId = null;

        if (typeId is null)
        {
            selectedDirection = 0;
            return;
        }

        var code = typesRaw.FirstOrDefault(t => t.Id == typeId)?.Code;
        selectedDirection = code switch
        {
            "OPENING" or "PURCHASE" or "RETURN" or "ADJ_IN" => +1,
            "ISSUE" or "WRITEOFF" or "ADJ_OUT" => -1,
            _ => 0,
        };
    }

    private async Task SaveAsync()
    {
        if (itemId is null || typeId is null || movedOn is null || quantity <= 0)
        {
            Notification.Notify(NotificationSeverity.Warning, "Missing fields",
                "Pick an item, type, date, and a quantity greater than zero.");
            return;
        }

        // Mutually-exclusive recipient guard mirrors the server-side check.
        var recipients = new[] { studentId.HasValue, schoolClassId.HasValue,
userId.HasValue }
            .Count(b => b);
        if (recipients > 1)
        {
            Notification.Notify(NotificationSeverity.Warning, "Pick one recipient",
                "Choose at most one of pupil / class / staff member.");
            return;
        }

        saving = true;
        try
        {
            var result = await MovementService.RecordAsync(new RecordStockMovementRequest
            {
                StoreItemId = itemId.Value,
                StockMovementTypeId = typeId.Value,
                MovedOn = DateOnly.FromDateTime(movedOn.Value),
                Quantity = quantity,
                UnitCost = unitCost,
                Reference = reference,
                Notes = notes,
                ReceivedFromSupplierId = supplierId,
                IssuedToStudentId = studentId,
                IssuedToSchoolClassId = schoolClassId,
```

```
                IssuedToUserId = userId,
            });

            if (result.Succeeded)
            {
                Notification.Notify(NotificationSeverity.Success, "Saved",
                    "Movement recorded.");
                Nav.NavigateTo($"/store/items/{itemId.Value}");
            }
            else
            {
                Notification.Notify(NotificationSeverity.Error, "Failed",
                    string.Join("; ", result.Errors));
            }
        }
        finally
        {
            saving = false;
        }
    }
}
```

9.7 Suppliers.razor

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Inventory/Suppliers.razor

```
@page "/store/suppliers"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher},
{Roles.SchoolStoreKeeper}")]
@rendermode InteractiveServer

@inject ISupplierService SupplierService
@inject NavigationManager Nav
@inject DialogService Dialog
@inject NotificationService Notification

<PageTitle>Suppliers · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>Suppliers</h1>
        <p>The vendors the school buys from. Each row tracks total purchases recorded
against the supplier.</p>
    </div>
    <div class="nps-page-actions">
        <RadzenButton Text="New supplier" Icon="add" ButtonStyle="ButtonStyle.Primary"
                      Click="@(() => OpenForm(null))" />
    </div>
</div>

<RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
    <div class="nps-filter-bar">
        <div class="nps-field">
            <label>Search</label>
```

```
            <RadzenTextBox @bind-Value="search" Placeholder="Name, contact, phone, email"
                           Change="@(_ => LoadAsync())" Style="min-width: 280px;" />
        </div>
        <div class="nps-field">
            <label>Status</label>
            <RadzenDropDown TValue="bool?" Data="@statusOptions" TextProperty="Text"
ValueProperty="Value"
                            @bind-Value="isActive" AllowClear="true" Placeholder="All"
                            Change="@(_ => LoadAsync())" Style="min-width: 160px;" />
        </div>
    </div>
</RadzenCard>

<RadzenCard class="nps-card">
    <RadzenDataGrid TItem="SupplierDto" Data="@suppliers" IsLoading="@loading"
                    AllowPaging="true" PageSize="25" EmptyText="No suppliers match your
filters.">
        <Columns>
            <RadzenDataGridColumn TItem="SupplierDto" Title="Name">
                <Template Context="s">
                    <strong>@s.Name</strong>
                    @if (!string.IsNullOrEmpty(s.ContactName))
                    {
                        <br />
                        <small style="color: var(--nps-ink-500);">@s.ContactName</small>
                    }
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="SupplierDto" Title="Contact" Sortable="false">
                <Template Context="s">
                    <div style="display:flex; flex-direction:column;">
                        <span>@s.Phone</span>
                        <small style="color: var(--nps-ink-500);">@s.Email</small>
                    </div>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="SupplierDto" Title="Purchases" Width="120px"
Property="PurchaseCount" />
            <RadzenDataGridColumn TItem="SupplierDto" Title="Total" Width="160px">
                <Template
Context="s"><strong>@s.TotalPurchased.ToString("N2")</strong></Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="SupplierDto" Title="Status" Width="120px">
                <Template Context="s">
                    @if (s.IsActive)
                    {
                        <RadzenBadge Text="Active" BadgeStyle="BadgeStyle.Success"
Variant="Variant.Flat" />
                    }
                    else
                    {
                        <RadzenBadge Text="Inactive" BadgeStyle="BadgeStyle.Secondary"
Variant="Variant.Flat" />
                    }
                </Template>
```

```
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="SupplierDto" Title="Actions" Width="200px"
Sortable="false"
                                  TextAlign="TextAlign.Right">
                <Template Context="s">
                    <div class="nps-inline-actions">
                        <RadzenButton Icon="edit" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                      ButtonStyle="ButtonStyle.Light" title="Edit"
                                      Click="@(() => OpenForm(s))" />
                        @if (s.IsActive)
                        {
                            <RadzenButton Icon="block" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                          ButtonStyle="ButtonStyle.Warning"
title="Deactivate"
                                          Click="@(() => SetActiveAsync(s, false))" />
                        }
                        else
                        {
                            <RadzenButton Icon="check_circle" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                          ButtonStyle="ButtonStyle.Success"
title="Activate"
                                          Click="@(() => SetActiveAsync(s, true))" />
                        }
                        <RadzenButton Icon="delete" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                      ButtonStyle="ButtonStyle.Danger" title="Delete"
                                      Click="@(() => DeleteAsync(s))" />
                    </div>
                </Template>
            </RadzenDataGridColumn>
        </Columns>
    </RadzenDataGrid>
</RadzenCard>

@if (showForm)
{
    <RadzenCard class="nps-card" Style="margin-top:1rem;">
        <h3 style="margin: 0 0 1rem;">@(editingId.HasValue ? "Edit supplier" : "New
supplier")</h3>
        <RadzenTemplateForm TItem="SupplierFormModel" Data="@form" Submit="@SaveAsync">
            <DataAnnotationsValidator />
            <div class="nps-form-grid">
                <div class="nps-field nps-span-2">
                    <label>Name *</label>
                    <RadzenTextBox @bind-Value="form.Name" />
                    <ValidationMessage For="() => form.Name" />
                </div>
                <div class="nps-field">
                    <label>Contact name</label>
                    <RadzenTextBox @bind-Value="form.ContactName" />
                </div>
                <div class="nps-field">
```

```
                    <label>Phone</label>
                    <RadzenTextBox @bind-Value="form.Phone" />
                </div>
                <div class="nps-field nps-span-2">
                    <label>Email</label>
                    <RadzenTextBox @bind-Value="form.Email" />
                </div>
                <div class="nps-field nps-span-2">
                    <label>Address</label>
                    <RadzenTextArea @bind-Value="form.Address" Rows="2" />
                </div>
                <div class="nps-field nps-span-2">
                    <label>Notes</label>
                    <RadzenTextArea @bind-Value="form.Notes" Rows="2" />
                </div>
            </div>
            <div class="nps-form-actions">
                <RadzenButton Text="Cancel" ButtonType="ButtonType.Button"
                              ButtonStyle="ButtonStyle.Light" Variant="Variant.Flat"
                              Click="@(() => showForm = false)" />
                <RadzenButton Text="@(saving ? "Saving..." : "Save")"
                              Icon="save" ButtonType="ButtonType.Submit"
                              ButtonStyle="ButtonStyle.Primary" Disabled="@saving" />
            </div>
        </RadzenTemplateForm>
    </RadzenCard>
}

@code {
    private IReadOnlyList<SupplierDto> suppliers = [];
    private bool loading;
    private bool saving;
    private bool showForm;
    private Guid? editingId;

    private string? search;
    private bool? isActive;

    private record StatusOption(string Text, bool? Value);
    private readonly List<StatusOption> statusOptions =
    [
        new StatusOption("Active", true),
        new StatusOption("Inactive", false),
    ];

    private SupplierFormModel form = new();

    private class SupplierFormModel
    {
        [System.ComponentModel.DataAnnotations.Required,
System.ComponentModel.DataAnnotations.StringLength(120)]
        public string Name { get; set; } = string.Empty;

        [System.ComponentModel.DataAnnotations.StringLength(120)]
        public string? ContactName { get; set; }
```

```
        [System.ComponentModel.DataAnnotations.StringLength(30)]
        public string? Phone { get; set; }

        [System.ComponentModel.DataAnnotations.EmailAddress,
System.ComponentModel.DataAnnotations.StringLength(256)]
        public string? Email { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(300)]
        public string? Address { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(500)]
        public string? Notes { get; set; }
    }

    protected override Task OnInitializedAsync() => LoadAsync();

    private async Task LoadAsync()
    {
        loading = true;
        try
        {
            suppliers = await SupplierService.ListAsync(new SupplierFilter
            {
                Search = search,
                IsActive = isActive,
            });
        }
        finally
        {
            loading = false;
            StateHasChanged();
        }
    }

    private void OpenForm(SupplierDto? s)
    {
        editingId = s?.Id;
        form = new SupplierFormModel
        {
            Name = s?.Name ?? string.Empty,
            ContactName = s?.ContactName,
            Phone = s?.Phone,
            Email = s?.Email,
            Address = s?.Address,
            Notes = s?.Notes,
        };
        showForm = true;
    }

    private async Task SaveAsync()
    {
        saving = true;
        try
        {
```

```
            OperationResult result = editingId.HasValue
                ? await SupplierService.UpdateAsync(new UpdateSupplierRequest
                {
                    Id = editingId.Value,
                    Name = form.Name,
                    ContactName = form.ContactName,
                    Phone = form.Phone,
                    Email = form.Email,
                    Address = form.Address,
                    Notes = form.Notes,
                })
                : await SupplierService.CreateAsync(new CreateSupplierRequest
                {
                    Name = form.Name,
                    ContactName = form.ContactName,
                    Phone = form.Phone,
                    Email = form.Email,
                    Address = form.Address,
                    Notes = form.Notes,
                });

            if (result.Succeeded)
            {
                Notification.Notify(NotificationSeverity.Success, "Saved",
                    $"Supplier '{form.Name}' saved.");
                showForm = false;
                await LoadAsync();
            }
            else
            {
                Notification.Notify(NotificationSeverity.Error, "Failed",
                    string.Join("; ", result.Errors));
            }
        }
        finally
        {
            saving = false;
        }
    }

    private async Task SetActiveAsync(SupplierDto s, bool active)
    {
        var result = await SupplierService.SetActiveAsync(s.Id, active,
CancellationToken.None);
        if (result.Succeeded)
        {
            Notification.Notify(NotificationSeverity.Success, "Updated",
                active ? $"'{s.Name}' activated." : $"'{s.Name}' deactivated.");
            await LoadAsync();
        }
        else
        {
            Notification.Notify(NotificationSeverity.Error, "Failed",
                string.Join("; ", result.Errors));
        }
```

```
    }

    private async Task DeleteAsync(SupplierDto s)
    {
        var confirm = await Dialog.Confirm(
            $"Delete '{s.Name}'? Only suppliers with no purchase history can be deleted.",
            "Delete supplier",
            new ConfirmOptions { OkButtonText = "Delete", CancelButtonText = "Cancel" });
        if (confirm != true) return;

        var result = await SupplierService.SoftDeleteAsync(s.Id);
        if (result.Succeeded)
        {
            Notification.Notify(NotificationSeverity.Success, "Deleted", $"'{s.Name}'
removed.");
            await LoadAsync();
        }
        else
        {
            Notification.Notify(NotificationSeverity.Error, "Failed",
                string.Join("; ", result.Errors));
        }
    }
}
```

10. Navigation, imports, and authorization

The previously-disabled 'Store & Inventory' nav placeholder is replaced with a six-item panel. The panel is wrapped in an AuthorizeView that grants access to SuperAdmin, HeadTeacher, and SchoolStoreKeeper.

Excerpt — NavMenu.razor

```
            <RadzenPanelMenuItem Text="Store & Inventory" Icon="inventory_2"
Expanded="true">
                <RadzenPanelMenuItem Text="Store dashboard" Icon="space_dashboard"
Path="/store" />
                <RadzenPanelMenuItem Text="Catalog" Icon="inventory_2"
Path="/store/items" />
                <RadzenPanelMenuItem Text="New item" Icon="add_box" Path="/store/items/new"
/>
                <RadzenPanelMenuItem Text="Movements" Icon="swap_vert"
Path="/store/movements" />
                <RadzenPanelMenuItem Text="Record movement" Icon="post_add"
Path="/store/movements/new" />
                <RadzenPanelMenuItem Text="Suppliers" Icon="local_shipping"
Path="/store/suppliers" />
            </RadzenPanelMenuItem>
        </Authorized>
    </AuthorizeView>
```

Excerpt — _Imports.razor

```
@using NaijaPrimeSchool.Application.Inventory
@using NaijaPrimeSchool.Application.Inventory.Dtos
```

11. Lifecycle of an inventory flow

11.1 Seeding the store

- Storekeeper opens /store/suppliers, clicks New supplier. Enters 'Lagos Books Ltd', a contact, a phone, an email.
- Opens /store/items, clicks New item. Enters 'English Textbook — Primary 4', picks category Books, unit Each, reorder level 5, opening quantity 60, opening unit cost ₦2,000. Saves.
- StoreItemService stages the StoreItem with QuantityOnHand = 60 plus an OPENING StockMovement for 60 × ₦2,000 in the same SaveChanges.
- Storekeeper lands on the item detail page, which now shows the opening movement and a stock value of ₦120,000.

11.2 Recording a purchase

- Lagos Books delivers 50 more copies at ₦2,100.
- Storekeeper opens /store/movements/new, picks the textbook item, picks Purchase as the type. The lower half flips to the inbound 'received from' panel.
- Picks Lagos Books Ltd as the supplier, enters quantity 50, unit cost 2,100, reference 'PO-2026-009'.
- Saves. StockMovementService writes the row, increments QuantityOnHand by 50 (now 110), and updates LastUnitCost to 2,100. Movement number is auto-generated as NPS/STK/<year>/<seq>.

11.3 Issuing to a class

- The Primary 4A teacher requisitions 38 textbooks (one per pupil).
- Storekeeper records a movement, picks Issue. The lower half flips to the outbound 'issued to recipient' panel.
- Picks Primary 4A as the class. Enters quantity 38, leaves unit cost blank (cost only matters for inbound movements).
- Saves. QuantityOnHand drops to 72.

11.4 Catching a low-stock alert

- Over the term, 65 more textbooks are issued. QuantityOnHand now sits at 7.
- Dashboard's 'Below reorder level' tile flips to 1.
- The Low-stock panel surfaces the textbook with a Low badge and a click-through to the item detail.
- Storekeeper raises a fresh purchase, runs through the same Purchase movement flow, and the badge clears.

11.5 Reversing a mistake

- Storekeeper realises a Purchase was keyed against the wrong item.
- Opens the movement log, finds the row, soft-deletes it.
- StockMovementService reads the row, multiplies Direction by Quantity, subtracts that from QuantityOnHand on the affected item, then soft-deletes the row.
- The audit history shows the row as hidden (only IgnoreQueryFilters returns it); the item's on-hand quantity is back to what it was before the mistake.

12. Smoke-test walkthrough

12.1 Build, migrate, run

```
dotnet restore
dotnet build NaijaPrimeSchool.slnx
dotnet run --project src/NaijaPrimeSchool.Web
```

First run applies the StoreAndInventory migration and seeds the three new lookup tables. Sign in as the SuperAdmin (or a user in the SchoolStoreKeeper role).

12.2 Verify navigation and lookups

- The Store & Inventory nav panel is now an active six-item dropdown rather than a disabled placeholder.
- SELECT Name, Code FROM ItemCategories ORDER BY DisplayOrder; shows ten rows.
- SELECT Name, Code FROM UnitsOfMeasure ORDER BY DisplayOrder; shows ten rows.
- SELECT Name, Code, Direction FROM StockMovementTypes ORDER BY DisplayOrder; shows seven rows with the right +1 / -1 values.

12.3 End-to-end happy path

8. Store → Suppliers → New supplier. Save 'Lagos Books Ltd'.
9. Store → New item. Save 'English Textbook — Primary 4', Books, Each, reorder level 5, opening quantity 60 at ₦2,000.
10. Open the item detail. Confirm one Opening Balance movement and QuantityOnHand = 60.
11. Store → Record movement. Pick the textbook, Purchase, supplier Lagos Books, quantity 50 at ₦2,100. Save.
12. Item detail now shows two movements and on-hand 110.
13. Record an Issue movement to a class for quantity 38. On-hand drops to 72.
14. Store dashboard shows the textbook in the 'Recent movements' list, the stock value reflects (72 × 2,100), and items below reorder is zero.

12.4 Error paths

15. Try recording an Issue larger than the on-hand quantity. Refused.
16. Try recording an Issue with both a pupil and a class picked. Refused by the form.
17. Try deleting a supplier that has a purchase. Refused.
18. Try deleting an item that has movement history. Refused.
19. Soft-delete a movement and watch the parent item's on-hand quantity rewind by the right amount.

13. Troubleshooting and gotchas

13.1 'Cannot remove N — only M are on hand'

Sprint 7 refuses to let an outbound movement drive the running balance negative. Either record a fresh Purchase or Adjustment In first, or split the movement so the requested quantity matches what is on hand. The friendly error message names the on-hand figure.

13.2 'A movement can be issued to at most one recipient'

Both the UI and the service-layer guard against this. If the Save button silently refuses, double-check the recipient panel — only one of pupil / class / staff member should be selected.

13.3 'Cannot delete a supplier with purchase history'

Suppliers that have ever been linked to a purchase movement stay on file for the audit trail. Use Deactivate instead — deactivated suppliers no longer appear in the active-suppliers dropdown on the record-movement page, but their history is preserved.

13.4 'SKU already exists'

The filtered unique index on StoreItem.Sku ignores NULLs, so you can have any number of items without an SKU. If you insist on filling it in, the value has to be globally unique across active items.

13.5 Dashboard stock value disagrees with my expectations

StockValue is QuantityOnHand × LastUnitCost — a single-point estimate. If the school buys at varying unit costs (the 60 textbooks at ₦2,000 plus 50 at ₦2,100), the valuation uses the most recent unit cost (2,100), not a weighted average. This matches how most small schools value stock; a future enhancement could expose a weighted-average column.

13.6 The IdentityRole migration warning

Same pre-existing warning that has accompanied every sprint since sprint 1. Harmless at runtime.

14. Forward-compatibility, today

- Parent and student portals (later sprint) will read StudentId-tagged issuance movements to show 'items received this term' on a pupil's profile.
- Purchase Orders can grow into their own table with approvals; the Reference column on a StockMovement captures a PO number today and can later become a foreign key.
- Per-batch tracking (expiry, lot number) is additive — a future StoreItemBatch table sitting between StoreItem and StockMovement does not require changing existing rows.
- Linking issuances to bursar invoices (a uniform issued is also a uniform billed) is a one-column add to StockMovement.
- Stock-take audits — periodic full counts that produce a stream of Adjustment In / Out movements — already work with the seeded types.

15. Appendix — files added or changed in sprint 7

Domain layer (new)	—
src/NaijaPrimeSchool.Domain/Inventory/ItemCategory.cs	Lookup.
src/NaijaPrimeSchool.Domain/Inventory/UnitOfMeasure.cs	Lookup.
src/NaijaPrimeSchool.Domain/Inventory/StockMovementType.cs	Lookup with Direction.
src/NaijaPrimeSchool.Domain/Inventory/Supplier.cs	Vendor entity.
src/NaijaPrimeSchool.Domain/Inventory/StoreItem.cs	Catalog entry.
src/NaijaPrimeSchool.Domain/Inventory/StockMovement.cs	Audit row.
Domain layer (modified)	—
src/NaijaPrimeSchool.Domain/Family/Student.cs	Added StockIssuances.
src/NaijaPrimeSchool.Domain/Academics/SchoolClass.cs	Added StockIssuances.
Application layer (new)	—
src/NaijaPrimeSchool.Application/Inventory/Dtos/SupplierDtos.cs	Supplier DTOs.
src/NaijaPrimeSchool.Application/Inventory/Dtos/StoreItemDtos.cs	Catalog DTOs.
src/NaijaPrimeSchool.Application/Inventory/Dtos/StockMovementDtos.cs	Movement DTOs.
src/NaijaPrimeSchool.Application/Inventory/Dtos/StoreSummaryDtos.cs	Dashboard DTOs.
src/NaijaPrimeSchool.Application/Inventory/ISupplierService.cs	Supplier service contract.
src/NaijaPrimeSchool.Application/Inventory/IStoreItemService.cs	Catalog service contract.
src/NaijaPrimeSchool.Application/Inventory/IStockMovementService.cs	Movement service contract.
Application layer (modified)	—
src/NaijaPrimeSchool.Application/Users/ILookupService.cs	Added 5 new lookup methods.
Infrastructure layer (new)	—
src/NaijaPrimeSchool.Infrastructure/Services/SupplierService.cs	Supplier CRUD.
src/NaijaPrimeSchool.Infrastructure/Services/StoreItemService.cs	Catalog CRUD + opening balance.
src/NaijaPrimeSchool.Infrastructure/Services/StockMovementService.cs	Audit log + dashboard summary.
src/NaijaPrimeSchool.Infrastructure/Persistence/Migrations/20260516054730_StoreAndInventory.cs	EF migration adding 6 tables.
Infrastructure layer (modified)	—

src/NaijaPrimeSchool.Infrastructure/ DependencyInjection.cs	Registered 3 new services.
src/NaijaPrimeSchool.Infrastructure/ Persistence/ApplicationDbContext.cs	Added 6 DbSets, ConfigureInventory.
src/NaijaPrimeSchool.Infrastructure/ Persistence/DatabaseInitializer.cs	Seeded the 3 new lookup tables.
src/NaijaPrimeSchool.Infrastructure/Services/ LookupService.cs	Added 5 new lookup methods.
Web layer (new)	—
src/NaijaPrimeSchool.Web/Components/Pages/ Inventory/StoreDashboard.razor	Storekeeper summary.
src/NaijaPrimeSchool.Web/Components/Pages/ Inventory/StoreItems.razor	Catalog list + filters.
src/NaijaPrimeSchool.Web/Components/Pages/ Inventory/CreateStoreItem.razor	New item with opening balance.
src/NaijaPrimeSchool.Web/Components/Pages/ Inventory/StoreItemDetail.razor	Edit + movement history.
src/NaijaPrimeSchool.Web/Components/Pages/ Inventory/StockMovements.razor	Movement log.
src/NaijaPrimeSchool.Web/Components/Pages/ Inventory/RecordStockMovement.razor	Record movement with direction-aware panel.
src/NaijaPrimeSchool.Web/Components/Pages/ Inventory/Suppliers.razor	Supplier directory + inline form.
Web layer (modified)	—
src/NaijaPrimeSchool.Web/Components/ _Imports.razor	Added Inventory + Inventory.Dtos usings.
src/NaijaPrimeSchool.Web/Components/ Layout/NavMenu.razor	Replaced disabled Store & Inventory placeholder with full panel.
Tooling (new)	—
tools/generate_sprint7_guide.py	This document's generator.

— End of the Sprint 7 implementation guide. With the storekeeper's workspace alive, the next sprint can turn to the parent and student portals — the last role groups still looking at disabled navigation.